

ADAS: An Intelligent Real-Time Road Accident Detection & Alert System based on Deep Learning Analysis of CCTV Video Streams in Lipa City

Chapter 1

INTRODUCTION

Background of the Study

Road traffic injuries remain a major global public health and development concern. The World Health Organization (WHO) (2023b) states that approximately 1.19 million people die annually due to road traffic crashes, making it one of the leading causes of death worldwide, particularly among individuals aged 5 to 29 years. In addition to mortality, they stated that tens of millions sustain non-fatal injuries each year, many of which result in long-term disability. Beyond health consequences, they mentioned that road crashes impose significant economic burdens, with global losses estimated at approximately 3% of most countries' gross domestic product due to medical costs, productivity losses, and infrastructure damage. These figures illustrate that road traffic accidents are not only a safety issue but also a substantial socioeconomic challenge.

The immediate period following a crash is crucial for both survival and recovery. WHO (2023b) also emphasizes that timely detection and rapid access to emergency care are essential elements of effective post-crash response systems. Recent analysis of national data from 2016 to 2020 in Saudi Arabia has documented variability in emergency medical response times, highlighting disparities in how quickly trauma patients receive care after injury (Alslamah et al., 2023). These findings reinforce the necessity of minimizing the interval between an accident and emergency response. Delays in accident notification reduce the opportunity for prompt medical intervention, which may negatively affect injury outcomes and overall survival.

In response, numerous countries have integrated advanced surveillance and intelligent transportation technologies into urban traffic management systems. For example, in 2025, Dubai implemented a fully autonomous artificial intelligence (AI) traffic monitoring system capable of detecting road violations and incidents in real time without direct human intervention, illustrating operational reliance on AI for roadway surveillance (Times of India, 2025). Similarly, India deployed an AI-powered traffic management system along the Dwarka Expressway, designed to automatically identify various categories of traffic violations through camera analytics and sensor integration (The Economic Times, 2025). In the United States, cities and states are increasingly adopting AI-enabled camera systems and roadway analytics tools to enhance road safety monitoring and infrastructure oversight (Inquirer.net, 2025). Collectively, these initiatives indicate a global transition from exclusively manual monitoring to technology-assisted detection mechanisms that reduce dependence on human observation.

In the Philippines, road traffic injuries continue to pose a significant concern. They represent the leading cause of mortality among Filipino youth aged 15 to 29 and are a major contributor to child fatalities (WHO, 2023a). Consequently, national agencies have initiated efforts to modernize surveillance infrastructure. The Metropolitan Manila Development Authority has announced plans to implement AI-enabled video detection systems and sensor-based traffic-monitoring technologies to enhance traffic management and incident detection (Philippine News Agency, 2024). Other local government units have similarly invested in smart traffic signalization systems integrated with closed-circuit television networks to improve real-time monitoring and enforcement operations (MindaNews, 2025). These efforts indicate a gradual incorporation of intelligent surveillance technologies into traffic management systems in selected Philippine cities. Nevertheless, the level of automation in accident detection remains minimal or absent nationwide.

Lipa City exhibits a similar situation. The Lipa City Disaster Risk Reduction and Management Office (Lipa CDRMO) manages 418 CCTV cameras distributed across major

roadways and intersections, forming the city's primary visual monitoring infrastructure for surveillance, including vehicular accidents. Despite this extensive network, monitoring remains predominantly manual. Operators must observe hundreds of camera feeds simultaneously across three eight-hour shifts. Given the scale of the camera network relative to available personnel, consistent real-time awareness of every vehicular accident cannot be guaranteed through manual observation alone. The presence of surveillance hardware does not ensure immediate situational awareness in the absence of automated detection mechanisms.

Accident data from Lipa CDRRMO records spanning 2023 to February 2026 underscore the prominence of vehicular accidents in the city's emergency response landscape. These records detail incident types, actions taken, demographic information of those involved, and locations. During this period, 3,219 emergency accidents were documented, with 1,858 classified as vehicular accidents; the remainder included self-accidents, medical emergencies, and incidents such as stabbing, beating, and trauma. Vehicular accidents thus accounted for approximately 57 percent of all emergencies managed. Annual figures show an increase from 379 vehicular accidents in 2023 to 626 in 2024, a 65.2 percent rise, followed by 732 incidents in 2025, representing a further 16.9 percent increase. Although preliminary data for 2026 indicate fewer cases, the upward trend from 2023 to 2025 highlights the escalating burden of road traffic incidents in the city.

Spatial analysis of Lipa CDRRMO records from 2025 to 2026 reveals geographic clustering of vehicular accidents. The top five barangays, namely Inosluban, Tambo, Dagatan, Antipolo del Norte, and Marawoy L.C., collectively account for approximately 21.9 percent of recorded vehicular accidents during this period. Inosluban alone comprised 9.1 percent of cases, followed by Tambo at 5.0 percent, Dagatan at 3.5 percent, Antipolo del Norte at 2.5 percent, and Marawoy L.C. at 1.8 percent. This concentration of incidents in specific localities indicates the presence of high-risk zones where improved real-time detection mechanisms could enhance emergency awareness and response coordination.

Beyond incident volume, the current reporting pipeline introduces variability in detection timelines. When asked about the typical duration for accident reports to reach the agency, Mr. Joseph Dondon Mitra, Head of the Operations and Warning Division of the Lipa CDRRMO, stated: “... *Malalaman lang kapag may nakakita at tumawag, kaya may delay bago makarating sa amin. Kumakain ng ilang minuto o segundo bago malaman ng TMD at maitawag sa amin ang insidente. May mga kaso na umaabot ng mga 5 minutes ang delay, depende kung may makakita agad at makapag-report. May pagkakataon na nalalaman na lang namin kapag tapos na ang insidente o na-respondehan na ng barangay nang walang tawag sa amin. Kapag nakita sa CCTV, mas mabilis ang response dahil agad itong namo-monitor.*”

This implies that accident reporting time depends on the specific circumstances of each case. Typically, a citizen reports the incident to the Traffic Management Division (TMD) or a police officer, who then relays the information to the agency. In rare instances, the report is communicated directly to them. Consequently, the interval between accident occurrence and official notification varies according to the initial observer and how the information is transmitted. Regardless of reporting circumstances, a significant delay will still occur before the CDRRMO is notified when reports have to come from external sources.

This variability highlights the manual nature of the current reporting process within the Lipa CDRRMO. The agency typically becomes aware of incidents only after they have been externally reported, rather than at the moment of occurrence. Given that vehicular accidents account for more than half of all recorded emergencies in Lipa City, dependence on manual monitoring and third-party reporting mechanisms likely contributes to avoidable response delays.

Although automated accident detection systems are technologically feasible and increasingly implemented internationally, such intelligent automation has not yet been integrated into the Philippines or Lipa City's surveillance framework. The primary gap is not the absence of surveillance infrastructure, but rather the lack of an automated, real-time accident detection

capability to support human operators. Implementing AI to address this gap offers the potential to reduce accident notification latency, improve situational awareness, and enhance the operational efficiency of emergency response in the local context.

Problem Statement

The current monitoring setup at Lipa CDRRMO is incapable of providing real-time accident detection for an extensive citywide network of cameras. This reliance on manual monitoring causes frequent, avoidable failures in accident detection, creating a critical time gap that delays emergency dispatch and directly compromises the survival and well-being of road accident victims. Specifically, this study aimed to solve the following problems:

1. The large volume of CCTV camera feeds makes it difficult for operators to continuously and effectively monitor all areas simultaneously, leading to missed or delayed detection of road accidents.
2. The absence of an automated alert system results in a heavy reliance on delayed accident reports from inter-agency endorsements, preventing the Lipa CDRRMO from being the primary and immediate responder to road emergencies.
3. The resulting delay in notification slows down emergency response times, leading to worsened injuries, increased property damage, and prolonged traffic congestion that could have been minimized with faster intervention.

Objectives of the Study

This study aimed to develop a real-time computer vision system that automated accident detection across Lipa CDRRMO's surveillance network. By transitioning from manual monitoring to AI-driven alerts, the project aimed to reduce reliance on delayed external accident reports

and to significantly reduce emergency dispatch latency, thereby improving public safety.

Specifically, this study aimed to:

1. To develop a custom-trained YOLO model capable of detecting vehicle-to-vehicle accidents in real-time across multiple camera streams, automating the surveillance process to overcome the limitations of manual monitoring.
2. To implement a real-time alert dashboard that delivers immediate visual and audible notifications to operators, enabling Lipa CDRMO to identify accidents independently without reliance on external inter-agency endorsements.
3. To evaluate the system's performance in terms of detection latency and accuracy, ensuring effective minimization of the notification gap and facilitating faster emergency intervention to mitigate injury severity and traffic congestion.

Scope and Delimitations

The project involved the development, evaluation, and edge-server deployment of a real-time road accident detection & alert system that accelerated emergency response times in Lipa City, Batangas, over a three-semester period (2025–2026). The functional and technical scope focused on a proof-of-concept software architecture that employed a custom-trained YOLO object detection model, optimized using a hybrid dataset comprising public repositories and archival CDRMO footage, to passively process standard RGB video feeds obtained via a MediaMTX RTSP proxy. Deployed on a dedicated edge inference server, the system integrated a Python worker process, a FastAPI backend, a local SQLite database, and a React dashboard. This interface enabled command center Operators to confirm or dismiss automated alerts through a HITL workflow, manage dynamic camera configurations, and monitor historical analytics, system health, and AI performance telemetry. Role-Based Access Control (RBAC) was enforced, with system Administrators limited to user account and security management.

System efficacy was quantified by measuring algorithmic detection accuracy, end-to-end edge processing latency, and operational workflow efficiency via HITL response times.

To prevent scope creep and maintain the stability of the city's active monitoring infrastructure, several clear boundaries were set. The software operated solely as an independent layer and did not integrate its user interface with the CDRRMO's Dahua DSS Video Management System, ensuring interoperability and uninterrupted city operations. All system testing and performance evaluations took place exclusively on researchers' independent hardware in a simulated environment to avoid disrupting active emergency monitoring. Maintenance, repair, network bandwidth optimization, and hardware scaling for Lipa City's 418-camera network were outside the scope of this study. The system did not include automated emergency vehicle dispatch routing or public-facing applications. The AI model was limited to detecting vehicle-to-vehicle collisions using standard optical video and did not use thermal or radar sensors or detect pedestrian accidents, traffic violations, theft, fire, or infrastructure damage. The platform was intended only for internal command center personnel and did not address the needs of on-ground traffic enforcers, other municipal departments, or neighboring city networks.

Significance of the Study

This study is significant to the following stakeholders:

Lipa CDRRMO Command Center Operators. The system's automated vehicle collision detection reduces cognitive overload associated with continuous video surveillance. This minimizes operator fatigue from manual monitoring across multiple screens and enables staff to transition from passive observation to active, effective dispatching.

First Responders and Partner Agencies. The system facilitates faster deployment for units such as local police, traffic management, and emergency medical services.

Immediate, verified visual evidence from Lipa CDRMO enables these agencies to coordinate efficiently and allocate appropriate resources promptly.

Accident Victims. The system addresses the urgent needs associated with severe medical trauma by reducing the notification gap from minutes to seconds. This improvement ensures that victims receive timely medical intervention during the critical period for survivability.

The Citizens of Lipa City. The system fosters a sense of security and strengthens civic trust within the community by actively safeguarding roadways. This improves quality of life and makes the city safer, more responsive, and a better place to live and commute.

The Local Government Units (LGUs). The study provides a scalable, cost-efficient model for smart city initiatives. It shows how local governments can improve public safety by retrofitting existing, passive CCTV networks with active AI, avoiding costly hardware replacements.

Future Researchers and Developers. The study establishes a technical foundation for deploying optimized single-stage computer vision models, such as the YOLO family, on localized edge servers. It serves as a reference for the development of real-time Human-in-the-Loop (HITL) surveillance systems in resource-constrained environments.

Chapter 3

METHODOLOGY

Introduction

This chapter presents the methodological approach adopted for conceptualizing, developing, and evaluating the real-time road accident detection & alert system. The discussion begins by establishing the overarching research design and the specific project development framework that informed the study. It then traces the system's lifecycle, starting with a comprehensive requirements specification that informs the foundational system architecture. The chapter proceeds to describe the development tools and technologies used to construct the software and AI model, as well as the testing and validation procedures implemented to ensure operational accuracy. This methodological plan concludes with the deployment and implementation of the completed system.

Research Design

This study adopts an applied research approach, driven by the need to address a concrete operational gap: the absence of automated vehicle collision detection within the emergency response workflow of the Lipa CDRMO. Rather than generating new theoretical frameworks, the study applies established computer vision techniques and modern software engineering practices to develop a functional, deployable system.

The project involved the development, evaluation, and edge-server deployment of a real-time road accident detection & alert system that accelerated emergency response times in Lipa City, Batangas, over a three-semester period (2025–2026). The functional and technical scope focused on a proof-of-concept software architecture that employed a custom-trained YOLO object detection model, optimized using a hybrid dataset comprising public repositories

and archival CDRRMO footage, to passively process standard RGB video feeds obtained via a MediaMTX RTSP proxy. Deployed on a dedicated edge inference server, the system integrated a Python worker process, a FastAPI backend, a local SQLite database, and a React dashboard. This interface enabled command center Operators to confirm or dismiss automated alerts through a HITL workflow, manage dynamic camera configurations, and monitor historical analytics, system health, and AI performance telemetry. RBAC was enforced, with system Administrators limited to user account management. System efficacy was quantified by measuring algorithmic detection accuracy, end-to-end edge processing latency, and operational workflow efficiency via HITL response times.

To prevent scope creep and maintain the stability of the city's active monitoring infrastructure, several clear boundaries were set. The software operated solely as an independent layer and did not integrate its user interface with the CDRRMO's Dahua DSS Video Management System, ensuring interoperability and uninterrupted city operations. All system testing and performance evaluations took place exclusively on researchers' independent hardware in a simulated environment to avoid disrupting active emergency monitoring. Maintenance, repair, network bandwidth optimization, and hardware scaling for Lipa City's 418-camera network were outside the scope of this study. The system did not include automated emergency vehicle dispatch routing or public-facing applications. The AI model was limited to detecting vehicle-to-vehicle collisions using standard optical video and did not use thermal or radar sensors or detect pedestrian accidents, traffic violations, theft, fire, or infrastructure damage. The platform was intended only for internal command center personnel and did not address the needs of on-ground traffic enforcers, other municipal departments, or neighboring city networks.

Project Scope and Boundaries

A clearly defined project scope is essential to ensure that the system remains technically feasible within the project's resource and time constraints and that stakeholder expectations are appropriately managed. The following delineates what is explicitly included and excluded from the scope of this study.

What Is Included (In Scope)

- **Functional Scope:** The system will continuously ingest and analyze live RTSP video streams from cameras deployed within the Lipa CDRRMO's existing infrastructure. Upon detecting a vehicle-to-vehicle collision, the system will automatically capture a visual snapshot with a bounding box, store an incident record in the local database with an Unverified status, render an active alert on the operator dashboard, and trigger a distinct audible alarm. The system will expose a Human-in-the-Loop (HITL) verification workflow that requires operators to select either Confirm or Dismiss for each generated alert, along with mechanisms for managing the complete incident lifecycle through Ongoing, Resolved, and Dismissed statuses. Additional functional capabilities include Role-Based Access Control (RBAC) for Operator and Administrator roles, camera configuration management, system health and hardware telemetry monitoring, AI performance tracking, accident analytics with data visualization, incident log export in CSV and PDF formats, and a searchable Help Center.
- **Technical Scope:** The system is implemented using a client-server architecture hosted entirely on a single on-premises Edge Inference Server. The server-side stack consists of a custom-trained YOLO object detection model executed via the

Ultralytics framework with OpenCV for RTSP frame capture, a FastAPI backend providing asynchronous REST API and WebSocket endpoints, and a SQLite relational database configured with Write-Ahead Logging (WAL). The client-side consists of a React-based Single Page Application using TypeScript, Zustand for state management, TanStack Query for data fetching, and Recharts for data visualization, styled with Tailwind CSS and shadcn/ui. Communication between the server and clients uses HTTPS for standard API requests and WebSockets for real-time alert delivery.

- **Data Scope:** The YOLO detection model will be trained on a hybrid dataset combining publicly available vehicular accident image repositories and targeted archival CCTV footage obtained from the Lipa CDRMO, curated and annotated using the Roboflow platform. The dataset is specifically augmented to represent the range of lighting, weather, and traffic conditions characteristic of the camera deployment locations within Lipa City, including nighttime glare, rain, and varying intersection geometries.
- **Evaluation Scope:** System performance will be evaluated against four primary criteria: detection accuracy, as measured by the mean average precision (mAP) of the trained YOLO model on a held-out test set; alert latency, defined as the time elapsed from AI detection to dashboard alert delivery, with a target ceiling of two seconds as specified in NFR-03; inference speed, targeting a minimum of ten to fifteen frames per second per camera stream; and operational effectiveness, specifically the reduction in the notification gap as measured against the fifteen-second detection-to-dispatch target in NFR-06.

- **Temporal Scope:** The project is scheduled for implementation over three academic semesters spanning the 2025-2026 and 2026-2027 school years, with all development, testing, and deployment activities conducted within this timeframe.
- **User Scope:** The system is designed exclusively for use by Lipa CDRRMO command center personnel. Two user roles are defined: Operators, who are responsible for monitoring live feeds, verifying incident alerts, managing the incident lifecycle, and camera configurations; and Administrators, who inherit all Operator privileges and additionally manage user accounts.
- **Geographic and Domain Scope:** The deployment focuses on traffic intersections and major roadways within Lipa City that are currently monitored by cameras already integrated into the CDRRMO's existing network, specifically including deployment points such as Crossing-Banaybanay, Air Base Intersection, Lipa Town Center-Tambo, Petron Banaybanay, and Unitop Star Tollway, among others. The system is developed as a proof-of-concept designed for future scalability, with the understanding that full deployment across the agency's 418-camera network is currently constrained by the available edge server hardware.

What Is Excluded (Out of Scope)

- The AI detection model is trained exclusively to recognize vehicle-to-vehicle collisions. The detection of pedestrian accidents, traffic rule violations, theft, fires,

infrastructure damage, or any other event category is explicitly outside the scope of the current model and will not be addressed in this study.

- This project operates as a software layer dependent on the CDRRMO's existing physical CCTV infrastructure. The maintenance, repair, optimization, or management of the physical IP cameras, Power over Ethernet (PoE) switches, and network bandwidth are considered external operational constraints and are not addressed within the scope of this study.
- To prevent interoperability issues with the proprietary Dahua DSS surveillance platform currently in use by the agency and to safeguard the continuity of the CDRRMO's critical 24/7 monitoring operations, the system will function as a fully standalone application with no integration into the existing Dahua DSS platform.
- The system's functional boundary ends at alert generation and operator verification. The automation of emergency vehicle dispatching or the coordination of inter-agency response protocols is explicitly excluded, as the CDRRMO's established emergency response procedures for these activities are already operational and effective and therefore fall outside the scope of this study.
- All performance evaluations and testing procedures will be conducted on the research team's independent hardware to simulate the live deployment environment without disrupting the CDRRMO's active monitoring operations.
- The web dashboard, incident logs, and system analytics will not be accessible to the general public, external agencies, or any network entity outside the CDRRMO's localized command center LAN. No data will be transmitted to or stored on any external cloud service.

Project Development Model

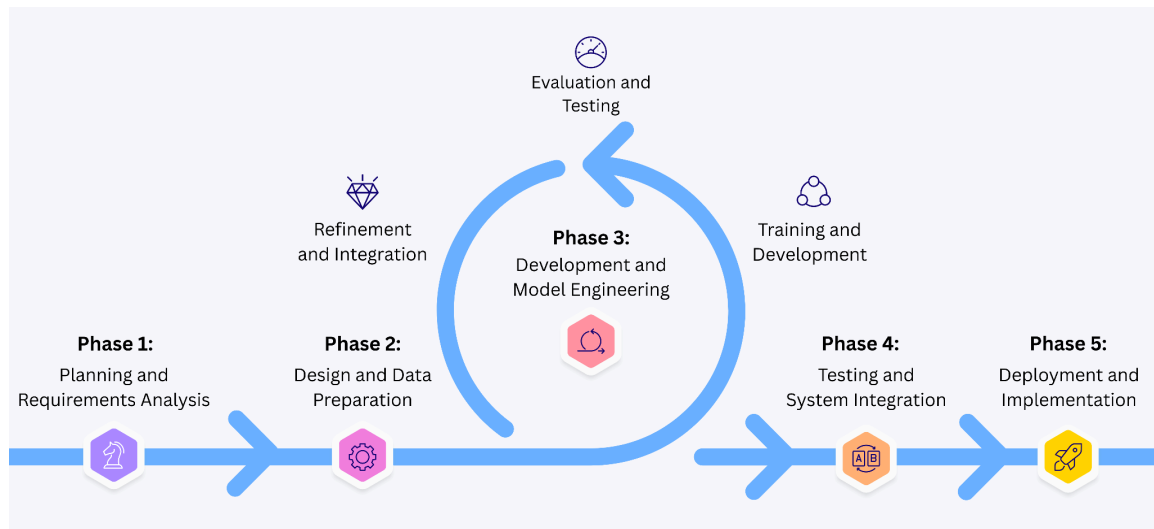


Figure 1.0. Hybrid Dual-Track Framework Diagram

The system comprises two distinct technical domains: an AI detection engine and a web application. Because both components require clear initial boundaries, flexible development phases, and strict concluding evaluations, the study implements a Hybrid Dual-Track Development Framework.

Rather than contrasting the two disciplines, several development frameworks were considered before selecting the Hybrid Dual-Track Development Framework. The Waterfall model provides a clear sequential process, making it suitable for projects with fixed requirements. However, it was not fully sufficient for this study because several components required repeated technical refinement after initial development, including the AI model, dashboard interface, backend APIs, database operations, WebSocket alert delivery, and system integration.

A purely Agile approach was also considered, but it was not selected because Agile relies on frequent stakeholder feedback and changing requirements across iterations. In this study, the core requirements were already established early through consultation with the Lipa CDRRMO. The system needed a stable direction from the beginning because it involved emergency response workflows, role-based access control, database structure, security rules, and deployment boundaries. Constantly revising requirements every sprint would have been unnecessary and could have disrupted the fixed operational scope of the project.

The Hybrid Dual-Track Development Framework was selected because it allowed the proponents to maintain a planned and stable project direction while still supporting iterative improvement during development. The AI track required repeated dataset preparation, model training, validation, error analysis, and retraining. At the same time, the software track also required iterative refinement of the React dashboard, FastAPI backend, SQLite database operations, camera management, incident logging, report generation, and real-time alert workflow. These iterations were not caused by changing client requirements, but by the need to test, improve, and stabilize each technical component.

Therefore, the Hybrid Dual-Track Development Framework best fits the proposed system because it balances fixed requirements with controlled iteration. The framework preserved the finalized operational scope of the Lipa CDRRMO while allowing both the AI detection engine and the web-based command center application to be refined through testing and integration. This approach supports the development of a reliable, secure, and real-time AI-assisted accident detection system.

Track 1: Hybrid Machine Learning Development Life Cycle.

Track 1 applies this structure to the YOLO AI object detection model. The core development of the model is inherently iterative because performance variables, such as dataset quality, environmental conditions, camera perspectives, and edge-hardware constraints, only become fully apparent after initial training. An early model iteration might perform optimally in daylight but fail under nighttime glare, requiring targeted dataset curation, augmentation and retraining. The ML DLC formalizes this feedback loop (train, evaluate, analyze failures, refine) until the AI satisfies the established performance thresholds.

Track 2: Hybrid Software Development Life Cycle.

Concurrently, Track 2 applies the exact same hybrid structure to the system software. During the initial predictive stage, system requirements are formally defined through consultation with Lipa CDRMO stakeholders. Because the system's core objectives are highly specific and clearly articulated from the outset, the proponents were able to lock in a definitive architectural blueprint without needing continuous requirement revisions. Following this predictive phase, development enters an iterative stage where the React dashboard, FastAPI backend, SQLite database, and AI engine are implemented, tested, and refined incrementally against the locked specifications.

Development is structured into five sequential phases. Both tracks progress in parallel, with each executing tasks aligned to its methodology.

Phase 1: Planning and Requirements Analysis.

Track 1 (MLDC) defines the model's target detection parameters, formally establishing an 85% mAP baseline to optimize real-time edge inference. This phase simultaneously evaluates the physical constraints of the Lipa City CCTV network, such as variable lighting, frame rates, and camera angles, and sets the corresponding data collection and annotation protocols required to train the model within these real-world limitations.

Track 2 (SDLC) involves gathering and formally documenting requirements through interviews and brainstorming sessions with Lipa CDRRMO command center personnel, resulting in a complete functional and non-functional requirements specification. The client-server architecture for the system has been defined, and the operational emergency protocols for the HITL alert workflow and RBAC roles have been finalized.

Phase 2: Design and Data Preparation.

Track 1 (MLDLC) begins its first data preparation cycle by curating a hybrid training dataset that combines publicly available vehicular accident image repositories with archival CCTV footage from the Lipa CDRRMO. This is followed by systematic manual bounding-box annotation and the application of image augmentation techniques such as brightness adjustment, rotation, and cropping to reflect the nighttime glare, rain, and varying camera angles typical of Lipa City traffic conditions.

Track 2 (SDLC) completes the system design artifacts required before development begins. The database schema is finalized in 3NF, the full suite of API endpoints is specified, and UI/UX wireframes for all operator and administrator dashboard views are produced and approved.

Phase 3: Development and Model Engineering.

Track 1 (MLDC) executes its core experimental loop. The YOLO model architecture is configured and trained on the prepared hybrid dataset, and is subsequently evaluated against the strict operational targets established in Phase 1: an 85% mean Average Precision (mAP) for classification fidelity and an Intersection over Union (IoU) threshold of ≥ 0.50 for spatial localization. Failure modes identified during evaluation, such as degraded detection under nighttime glare or occluded vehicles, prompt targeted dataset curation, augmentation, and retraining. This iterative train-evaluate-refine cycle continues strictly until the model successfully

breaches the predefined 85% mAP and 0.50 IoU thresholds. Once these metrics are validated, the model weights are locked, and the inference engine is exported and optimized for deployment on the localized edge server to ensure it meets the low-latency processing requirements of the command center's Human-in-the-Loop workflow.

Track 2 (SDLC) transitions into active development. Developers iteratively implement and test individual software modules against the specifications established in Phases 1 and 2. Each module is built, unit-tested, and integrated incrementally.

Phase 4: Testing and System Integration.

In this phase, the parallel development lifecycles formally converge for rigorous end-to-end validation. With the YOLO model weights finalized and optimized, it is integrated directly into the AI engine and FastAPI backend. The fully assembled software architecture is then subjected to comprehensive system testing within a simulated LAN environment. Rather than evaluating static datasets, the system processes simulated live RTSP video feeds to validate real-time inference stability, track continuous hardware resource utilization, and ensure that AI detections successfully propagate through the SQLite database to trigger sub-15-second WebSocket alerts on the React dashboard. This quality assurance process also verifies the complete HITL resolution workflow and strictly enforces RBAC routing between Operators and Administrators. Finally, the phase concludes with formal UAT, allowing Lipa CDRMO dispatchers to interact with the system in a staging environment to validate UI/UX efficiency, operational readiness, and the successful fulfillment of all defined project requirements prior to physical deployment.

Phase 5: Deployment and Implementation.

The parallel development tracks formally unify as the finalized software architecture is deployed to the physical edge-inference server. The hardware is permanently installed within

the Lipa CDRRMO command center and integrated securely into the agency's existing CCTV VLAN. Operating as a headless client, the system is configured to passively ingest video substreams directly from the proprietary Dahua VMS. The implementation executes a phased rollout strategy, beginning with a pilot deployment at high-risk intersections to monitor real-world hardware telemetry before incrementally scaling to integrate the entirety of the agency's 418-camera network. The lifecycle concludes with the formal handover of the system, which includes comprehensive operational training for dispatchers, security management training for administrators, and the establishment of long-term AI retraining and hardware maintenance protocols.

Requirements Analysis

This section defines the Functional Requirements Specification (FRS), the Non-Functional Requirements Specification (NFRS), and the system's Use Cases. The requirements were developed through stakeholder interviews and iterative brainstorming sessions.

The requirements elicitation process began with an initial interview with the head of the Lipa CDRRMO operations and warning division to identify operational challenges and technological opportunities. Based on these initial insights, the development team conducted internal brainstorming sessions to conceptualize potential system features and solutions. Over several weeks, an iterative validation loop was implemented, involving follow-up interviews with the CDRRMO to clarify functionalities, refine the system scope, and ensure alignment with emergency response protocols. This cyclical process informed the finalized system requirements presented in the following subsections.

Functional Requirements Specification (FRS)

Table 2.0. Functional Requirements

FR-01 User Authentication	The system shall require all users to securely log in using a valid username and password before accessing the web dashboard.
FR-02 Role-Based Access Control (RBAC)	The system shall enforce two distinct access levels. Operators shall have operational access to the analytics dashboard, active camera status lists and configurations, detection logs, and system health and AI performance metrics. Administrators shall inherit all Operator privileges and have exclusive access to User Account Management.
FR-03 User Account Management	The system shall provide account management capabilities differentiated by access level. Administrators shall have global permissions to create, edit, and delete user accounts, assign roles, and change passwords within the User Management module, which shall include a dynamic search function for querying the user directory. Operators shall be limited to self-service profile management, permitting only updates to their own account details and personal passwords.
FR-04 Video Stream Ingestion	The system shall connect to and retrieve video feeds from the agency's CCTV network.
FR-05 Automated Collision Detection	The system shall continuously analyze active video feeds using an AI computer vision model to detect vehicle-to-vehicle collisions within each camera's field of view.
FR-06 Alert Generation and Snapshot Capture	Upon detecting a collision, the system shall automatically capture a visual snapshot with a bounding box around the incident, store the event record in the local database with an Unverified status, display the active alert on the web dashboard, and trigger an audible alarm to notify the operator.
FR-07 Audible Alert Timeout and Escalation	Along with the visual alert, the system shall include a 'Mute/Snooze' button to give the operator a quiet moment to verify the incident. However, if the alert is left 'Unverified' for too long (e.g., 30 seconds), the alarm will automatically sound again to ensure no emergency is missed.
FR-08 Alarm Configuration Module	The system shall provide a settings interface allowing users to select distinct auditory alert tones, control system volume, and define the temporal duration (in seconds/minutes) for the alert suppression (snooze) feature.
FR-09 Human-in-the	The system shall present the captured incident snapshot to the user, requiring selection of either Confirm (to verify a true accident) or Dismiss (to

-Loop (HITL) Verification Workflow	indicate a false positive) before clearing the alert from the primary dashboard.
FR-11 False Positive Handling	If the user selects Dismiss, the system shall update the incident database record to reflect a Dismissed status and clear the notification from the dashboard. To prevent redundant alerts from the same environmental noise, the system shall enforce a one-minute AI detection cooldown on the specific camera feed before resuming active monitoring.
FR-12 True Positive Handling	If the user selects Confirm, the system shall update the incident database record to reflect an Ongoing status. The system must keep the AI detection for the camera paused throughout the incident. AI detection shall resume for that feed only after an operator manually updates the record as Resolved (indicating a cleared emergency) or Dismissed (indicating an aborted or misidentified event), thereby preventing duplicate alerts for the ongoing incident.
FR-13 Incident Logging	The system shall maintain a structured, uniquely identified database record for each detected incident. Each record must include essential telemetry such as visual evidence, camera location, initial detection time, and AI confidence levels, and must be continuously updated to track the complete operational lifecycle. This audit trail shall log all status changes, the personnel responsible for verifying and resolving the incident, and the corresponding timestamps.
FR-14 Historical Logs Search and Filtering	The system shall provide a dynamic search interface that enables users to query and retrieve historical incident logs. Users shall be able to filter and sort data by parameters such as date range, camera name, current status, and associated operator.
FR-15 Camera Configuration Management	The system shall provide a camera management interface displaying real-time status indicators for each feed, reflecting both network connection status (e.g., Connected, Disconnected, Connecting, Unresponsive) and AI operational status (e.g., Active, Inactive, Paused, Unresponsive). Within this interface, users shall be able to manually add, edit, or remove camera configurations, fully disable a camera to halt network and AI activity, and use dynamic search and filtering to query the camera list by specific statuses.
FR-16 System Health and Hardware Telemetry	The system shall provide a System Health dashboard to continuously monitor server hardware utilization and AI processing stability. The interface must display real-time telemetry for system uptime, hardware resource consumption (e.g., CPU, GPU, RAM, and Disk Storage), and inference processing speeds, with historical filtering and sorting capabilities to enable administrators to track system health trends over time.

FR-17 AI Performance	The system shall provide an AI Performance interface to track historical detection efficacy and model accuracy. The interface must display both global and per-camera statistics on incident volumes, precision scores, and average AI confidence scores, providing the data needed to analyze edge cases and improve AI performance.
FR-18 Accident Analytics and Visualization	The system shall provide an analytical dashboard to aggregate and visualize historical accident data. The interface must display global Key Performance Indicators (KPIs) summarizing accident volumes and resolution statuses. The system shall also generate graphical reports illustrating temporal and spatial accident trends, with dynamic filtering and sorting capabilities.
FR-19 Report Generation and Data Export.	The system shall allow users to export filtered and sorted historical incident records, operational analytical summaries, and AI performance datasets into standardized file formats (e.g., CSV, PDF). These exports shall support external sharing, inter-agency coordination, official record-keeping, and off-site AI model retraining processes.
FR-20 Help Center	The system shall provide a centralized, searchable Help Center to support user onboarding and routine system navigation. This module must include comprehensive digital documentation tailored to each user's access level, including standard operating procedures (SOPs) for incident resolution, general dashboard navigation guides, and frequently asked questions (FAQs).
FR-21 Activity Audit Trail	The system shall maintain an activity audit trail for critical user actions that affect incident verification, camera availability, official reporting, and account management. For Operators, the system shall record accident confirmation, dismissal, resolution, terminal dismissal correction, camera addition, camera editing, camera enabling or disabling, camera removal, report generation, and incident record export. For Administrators, the system shall record user account creation, account editing, account disabling, role changes, and password updates. The system shall also record successful and failed login attempts for both user roles. Each audit record must include the user ID, role, action performed, affected record, timestamp, and action result.

Table 3.1. Performance Requirements

NFR-01 Algorithmic Accuracy	The AI inference engine shall maintain a minimum Mean Average Precision (mAP) of 85% and an Intersection over Union (IoU) threshold of ≥ 0.50 for detecting vehicle-to-vehicle collisions in real-world environmental conditions.
NFR-02 Inference Latency	The AI inference engine shall process live video frames and complete bounding box generation within 100 milliseconds per frame to support continuous monitoring.
NFR-03 Frame Rate Maintenance	The edge server shall process connected camera streams at a minimum of 10 to 15 frames per second (FPS), minimizing computational overhead while maintaining real-time incident detection.
NFR-04 Alert Response Time	The backend shall deliver a visual and audible incident alert, including the captured video snapshot, to the web dashboard within 2 seconds of collision detection by the AI model.
NFR-05 Telemetry Refresh Rate	To optimize edge server resource utilization and prevent database bloat, the system shall enforce a strict separation between live dashboard updates and historical telemetry logging. For live user interface viewing, the backend shall transmit real-time hardware and AI inference metrics to the System Health dashboard every 10 to 15 seconds using in-memory data. For historical chart generation, the system shall record high-resolution telemetry logs to the database at 5-minute intervals, with logs pruned after 48 hours. Aggregated, low-resolution trends shall be computed hourly and retained for a rolling 30-day period.
NFR-06 Report Generation Speed	The system shall generate and initiate the download of requested analytical reports and historical log exports within 5 seconds for datasets covering up to 30 days.

Table 3.2. Scalability Requirements

NFR-07 Hardware Scalability	The system architecture shall support vertical scaling, enabling the edge server to process additional concurrent camera streams by upgrading hardware without necessitating codebase modifications.
NFR-08 Database Performance	The local SQLite database shall be indexed and optimized to support querying and filtering up to 100,000 historical incident logs and analytical data points, ensuring dashboard load times do not exceed 3 seconds.

Table 3.3. Usability Requirements

NFR-09 Operational Efficiency (Detection-to- Dispatch)	The system shall reduce the notification gap by enabling operators to visually identify, verify, and initiate dispatch protocols for vehicular collisions within 15 seconds of the incident occurring on camera.
NFR-10 Workflow Efficiency	The web dashboard shall enable users to visually verify an incident and select either "Confirm" or "Dismiss" in three clicks or fewer.
NFR-11 Alert Distinctiveness	The system shall generate an audible notification for new incidents that is clearly distinguishable from ambient noise in a standard emergency command center.
NFR-12 Learnability	The dashboard user interface shall be sufficiently intuitive to allow a new operator with basic computer literacy to learn to process and verify an alert within a 15-minute training session.

Table 3.4. Reliability Requirements

NFR-13 System Availability	The system, including the AI engine and web dashboard shall maintain 99.9% uptime, excluding planned maintenance.
NFR-14 Network Fault Tolerance	If a camera stream disconnects due to network instability or power loss, the system shall automatically attempt to reconnect to the feed every 10 seconds.
NFR-15 Process Isolation	The failure, freezing, or manual termination of a single camera's AI detection shall be strictly isolated and shall not cause the main server, the React dashboard, or the processing threads of other active cameras to crash.
NFR-16 Daily System Restart	To ensure continuous long-term operation and mitigate gradual memory degradation, the system shall automatically restart every 24 hours during a configurable low-traffic window (e.g., 3:00 AM), completing the memory flush and recovery in under 10 seconds.
NFR-17: Asynchronous Alert Recovery	To ensure continuity of emergency response, the web dashboard shall automatically synchronize its state with the system backend upon every initial load or network reconnection. The system shall immediately retrieve and display all currently "Unverified" incident alerts.

NFR-18: Data Redundancy & Recovery	The system shall execute automated daily backups of the SQLite database without interrupting active AI inference or triggering database locks. Furthermore, the system shall provide an administrative recovery mechanism capable of restoring the database and resuming automated alerts within a 60-second operational window.
------------------------------------	--

Table 3.5. Security Requirements

NFR-19 Session Security	The system shall enforce secure, token-based authentication [e.g., JSON Web Tokens(JWT)] for all user accounts and maintain persistent sessions to ensure the dashboard remains active and ready to receive alerts during extended standby periods.
NFR-20 Data Localization	To ensure data privacy and compliance with government security standards, all data shall be stored and processed exclusively on the edge server's physical drives and shall not be transmitted to external cloud services.
NFR-21 Audit Trail Integrity	The system shall preserve audit trail records in an append-only format to support accountability and post-incident review. Operators and Administrators shall not be allowed to edit or delete audit records. Administrators shall only be allowed to review audit records for operational and security verification.

Table 3.6. Maintainability Requirements

NFR-22 Modular AI Upgrades	The system architecture shall enable developers to update or replace AI model weights without requiring modifications to the web application.
----------------------------	---

Use Cases

Use Case 1: Log In

Actor: Operator, Administrator

Preconditions:

- The system is active on the local network.
- The user has a registered account.

Main Flow:

1. The user navigates to the web dashboard login page.
2. The user enters their valid username and password.
3. The system validates the credentials against the local database.
4. The system determines the assigned role (Operator or Administrator).
5. The system logs the user in and loads the web app restricted to their clearance level.

Alternative Flows:

- 2a. Missing Input Field(s): If the user submits the form with an empty username or password field, the system terminates the login attempt, highlights the incomplete fields, and displays a validation message without querying the database.
- 3a. Invalid Credentials: The system does not find a match for the provided username and password combination. The system displays a generic error message and denies access.
- 3b. Soft-Deleted / Deactivated Account: The system successfully matches the credentials but determines that the account status is inactive. The system aborts the login process, displays a security message, and denies access.

Postconditions:

- A secure, persistent, timed session is established.
- The user can only access features permitted by their specific role.

Use Case 2: Manage User Accounts

Actor: Administrator

Preconditions:

- The system is active on the local network.
- The user is logged in with Administrator privileges.

Main Flow:

1. The Administrator navigates to the "User Management" panel.
2. The Administrator selects an action: Add New User, Edit User Details, Change Password, or Delete User.
3. The Administrator enters the required information into the corresponding form (e.g., new account details, updated roles, or new passwords) or confirms the deletion prompt.
4. The system validates the input to ensure all required fields are populated and formatted correctly.
5. The system processes the request and updates the corresponding database record.
6. The system displays a success confirmation message and refreshes the active user directory.

Alternative Flows:

- 4a. Username Already Exists (Add User): If creating a new user and the requested username is already taken, the system halts the process, highlights the field, and prompts the Administrator to choose a different username.
- 4b. Password Mismatch (Change Password): If the Administrator types two different passwords in the "New Password" and "Confirm New Password" fields, the system blocks the submission and requests correction.
- 5a. Primary Administrator Lockout (Edit/Delete): When rendering the User Management module or accessing a specific user's "Edit" modal, the system calculates the total number of active Administrators. If the target account is the last remaining active Administrator, the system disables the delete button and locks the "Role" radio buttons. A tooltip over the disabled elements states,

"Cannot modify or delete the final active Administrator," thereby preventing any
lockout action.

Postconditions:

- The user database is updated, and access rights (or revocations) are immediately enforced.
- Any user subjected to a "Delete" action is permanently locked out; however, their historical actions remain intact in the database for FRS-10 auditing purposes.

Use Case 3: Manage User Profile

Actor: Operator

Preconditions:

- The system is active on the local network.
- The user is logged in with Operator privileges.

Main Flow:

1. The user navigates to their profile.
2. The user selects an action: Edit User Profile (e.g., Username) or Change Password.
3. The user inputs the updated information.
4. The system validates the inputs, ensuring all fields are correctly formatted, the current password matches the database, and any new username is not already taken.
5. The system updates the user's specific row in the local database.
6. The system displays a success confirmation message and updates the UI to reflect any changed profile details.

Alternative Flows:

- 5a. Incorrect Current Password: The user fails the security authorization check by entering the wrong current password. The system blocks the change in password, highlights the field, and displays an error message
- 5b. Username Already Exists (Edit Details): The user attempts to change their username to one that is already active in the system. The system halts the update and displays an error message
- 5c. New Password Mismatch (Change Password): The user types two different passwords in the "New Password" and "Confirm New Password" fields. The system blocks the submission and requests correction.

Postconditions:

- The user database is updated, and access rights (or revocations) are immediately enforced.

Use Case 4: Manage Cameras

Actor: Operator, Administrator

Preconditions:

- The user is logged into the system.
- A physical CCTV camera is active on the local network and outputting a valid RTSP stream.

Main Flow:

1. The user navigates to the "Cameras" module.
2. The user selects a specific management action: Add New Camera, Edit Camera Configuration, Disable/Enable Feed, or Remove Camera.
3. If adding or editing, the user inputs the Camera Name and unique Channel ID.

4. The system concatenates the globally stored VMS Server credentials with the specific Channel ID into a standardized RTSP URL format and verifies that the constructed URL is not already assigned to an existing active camera.
5. The system executes a network handshake using the constructed URL, verifies stream integrity, and starts an isolated AI inference worker process for that feed.
6. If disabling or removing, the system gracefully terminates the active AI worker process for that feed, drops the network socket, and updates or deletes the database record.
7. The system updates the UI with the corresponding real-time status indicator and displays a success confirmation message.

Alternative Flows:

- 4a. Duplicate Configuration Variables (Error Prevention): If the user inputs a Camera Name that already exists, or if the system's concatenated RTSP URL matches an active camera in the database, the system terminates the configuration process. The system highlights the specific field(s) and displays an error message to prevent database conflicts and dashboard confusion.
- 5a. Stream Unreachable (Network Failure): If the system fails to connect to the RTSP URL during the handshake, the system saves the configuration and flags the status as Unresponsive. It terminates the AI worker for that feed and initiates the 10-second reconnect protocol without causing dashboard instability.

Postconditions:

- The camera database is updated to reflect the new configurations or deletions.
- Live streams are actively ingested with their own isolated AI processes, or gracefully shut down to free up server GPU/RAM resources.

Use Case 5: Verify Accident Alert

Actor: Operator, Administrator

Preconditions:

- The user is logged into the system.
- The system is actively ingesting a camera feed with an isolated AI inference worker process running.

Main Flow:

1. The system detects a vehicle collision and captures a visual snapshot featuring a drawn bounding box.
2. The system generates an initial accident record in the local database with an "Unverified" status (saving the time detected, camera name, snapshot, and AI confidence score).
3. The system sends the active alert to the user interface and triggers an audible alarm at the user's configured sound and volume level to notify the user.
4. The user clicks "Snooze." The system mutes the audible alarm for that specific camera's alert and begins a countdown timer based on the user's configured snooze duration. All other active or incoming alerts on other cameras remain unaffected.
5. The user reviews the snapshot and/or live feed to visually verify the accident.
6. The user selects "Confirm".
7. The system updates the database record with the status changed to "Ongoing," the user's name as "Verified by," and the exact "Time Verified" timestamp.
8. The system maintains the camera's AI detection in a Paused state to prevent redundant alerts during the emergency. The snooze re-alarm is suppressed as the alert has been resolved within the snooze window.

9. Once the physical emergency is cleared, the user manually updates the ongoing accident log to "Resolved".
10. The system updates the database record with the status changed to "Closed," the user's name as "Closed by," and the exact "Time Closed" timestamp, then immediately resumes active AI detection for that specific camera feed.

Alternative Flows:

- 4a. User Proceeds Without Snoozing: If the user does not click "Snooze" and proceeds directly to review, the audible alarm continues to sound until the alert is resolved via "Confirm" or "Dismiss."
- 4b. Snooze Timer Expires Before Resolution: If the configured snooze duration elapses and the alert has not been confirmed or dismissed, the system re-triggers the audible alarm for that specific camera's alert to prompt the user to act. The user may snooze again, after which a new countdown begins.
- 6a. Immediate False Positive: The user identifies the alert as a false positive and clicks "Dismiss." The system updates the record status to "Dismissed," logs the user's name, clears the user interface alert, enforces a 1-minute AI detection cooldown on that specific camera before resuming detection, and suppresses the snooze re-alarm as the alert has been resolved.
- 10a. Human Error Correction (Terminal Dismissal): The operator determines that the initial "Confirm" action was a human error and that the "Ongoing" event is not a collision. The user updates the record status to "Dismissed." The system updates the database record and resumes AI detection, allowing operators to correct verification mistakes.

Postconditions:

- The complete operational lifecycle of the incident is permanently saved as a single row in the local database for audit and reporting purposes.

- The camera's AI detection state (Paused vs. Active) is safely and predictably managed throughout the emergency's lifecycle.

Use Case 6: Monitor System Health and Performance

Actor: Operator, Administrator

Preconditions:

- The user is logged into the system.
- The edge server and AI processes are actively running.

Main Flow:

1. The user navigates to the "System Health" dashboard.
2. The system queries the local database to retrieve and render the default historical hardware trends (e.g., the last 48 hours) for CPU, GPU, RAM, and Core Temperatures on interactive line charts.
3. Every 10 to 15 seconds, the system receives a batched telemetry packet containing the rolling averages for hardware utilization, overall system uptime, average Inference Latency (ms), and Processing Speed (FPS).
4. The system dynamically updates live KPIs and appends new data points to line charts without requiring a full-page refresh.
5. The user clicks a time-series filter (e.g., "30-day Trend") to query older telemetry logs and visually analyze the server for long-term hardware degradation or memory leaks.

Alternative Flows:

- 3a. Telemetry Service Disconnect (Network Fault): If the dashboard loses connection to the backend telemetry service, the system freezes the UI charts, displays a non-intrusive "Telemetry Offline - Reconnecting..." warning banner, and silently attempts to re-establish the connection in the background.

- 4a. Critical Hardware Warning (Thermal/Usage Threshold): If the incoming batched telemetry packet indicates the CPU or GPU has crossed a critical operational threshold (e.g., Core Temp > 85°C, or RAM > 95%), the system dynamically highlights the affected metric card in red and displays a warning to prompt immediate administrative review.

Postconditions:

- The user is fully informed of the hardware and software stability of the edge server.
- System compute resources remain optimized for the AI inference engine due to the batched, low-overhead telemetry polling.

Use Case 7: Analyze AI Performance and Model Efficacy

Actor: Operator, Administrator

Preconditions:

- The user is logged into the system.
- The system database contains a repository of historical incident logs, including verified True Positives and user-dismissed False Positives.

Main Flow:

1. The user navigates to the "AI Performance" module.
2. The system queries the local database and mathematically computes global Machine Learning Operations KPIs, specifically the AI Precision Score, Accident Confidence Score, and Dismissed Confidence Score.
3. The system displays these global KPIs in top-level metric cards and populates a detailed data table below it, breaking down the exact Precision and Confidence scores on a strictly per-camera basis

4. The Administrator uses dynamic filtering tools (e.g., selecting a specific Date Range or a specific CCTV Camera) to isolate environmental edge cases (e.g., investigating why "Camera B" has a sudden spike in False Positives at night).
5. The user clicks "Export" to extract the data for off-site AI retraining.
6. The system compiles the analytical summary and historical logs into a standardized file format (e.g., CSV or PDF) and triggers the file download.

Alternative Flows:

- 2a. Insufficient Baseline Data (Zero-Division Prevention): If the system is newly deployed and lacks enough verified incidents to mathematically calculate a Precision Score, the system prevents backend calculation errors (divide-by-zero). It gracefully renders empty chart states with a placeholder message.
- 6a. Extended Report Generation (Async Loading): If the user queries an exceptionally large, multi-month dataset that cannot be processed instantaneously, the system prevents the UI from freezing. It displays a visual spinner and processes the export asynchronously.

Postconditions:

- The user successfully identifies real-world environmental edge cases where the AI struggles.
- * The extracted dataset is formatted for use by developers to retrain and improve the underlying YOLO or computer vision model weights.

Use Case 8: Analyze Incident Trends and Operational Analytics

Actor: Operator, Administrator

Preconditions:

- The user is logged into the system.
- The system database contains a repository of historical event records.

Main Flow:

1. The user navigates to the primary "Dashboard" panel.
2. The system queries the database to aggregate historical records of verified accidents (specifically those with "Ongoing" or "Resolved" status).
3. The system then calculates and displays the global KPIs: Pending, Total Accidents, and Total Resolved.
4. The system renders the high-level graphical reports, specifically an Accident Frequency by Location chart (horizontal bar graph) and a Peak Accident Times (24H) chart (line graph).
5. The user applies dynamic filters, selecting a specific Date Range or a specific Camera Name.
6. The system recalculates the metrics in real-time and updates the charts and KPI cards to reflect the newly filtered dataset.
7. The user clicks the "Export" button to export the currently displayed data to a standardized format (e.g., CSV or PDF).

Alternative Flows:

- 6a. Extended Report Generation (Async Loading): If the user queries an exceptionally large, multi-month dataset that cannot be processed instantaneously, the system prevents the UI from freezing. It displays a visual spinner and processes the export asynchronously.

Postconditions:

- The user gains a data-driven understanding of traffic accidents across Lipa City.
- The aggregated data is successfully exported for official CDRRMO reporting.

Use Case 9: Search and Review Historical Detections

Actor: Operator, Administrator

Preconditions:

- The user is logged into the system.
- The system database contains a repository of historical event records.

Main Flow:

1. The user navigates to the "Detections" module and selects the "Logs" tab.
2. The system performs a paginated database query and displays a table of historical event records, displaying high-level data (Accident No., Timestamp, Camera Name, Status, Last Handled by, Last Updated).
3. The user applies one or more dynamic filters (e.g., searching a specific Accident No., selecting a Camera Name, Date Range, Status, or specific Operator).
4. The system recalculates the query and updates the data table to display only the matching records.
5. The user selects the View option (eye icon) in the Actions column for a specific incident.
6. The system retrieves the complete database record and opens a detailed modal overlay.
7. The system displays the captured incident snapshot (retaining the AI bounding box), the complete core telemetry (AI-Confidence Score), and the operational Audit Trail (identifying the specific personnel and exact timestamps for both the Verification and Closure actions).
8. The user reviews the evidence and closes the modal to return to the filtered list.

Alternative Flows:

- 4a. Null Search Results: If the applied search query or filter combination yields no matching records, the system displays an empty-state graphic in the table area with the message: "No detection logs match the selected criteria."

- 8a. Export Filtered Logs: At any point while viewing the table, the user selects the Export button. The system compiles the currently filtered dataset into a standardized format (e.g., PDF or CSV) and initiates a download, using an asynchronous loading state if the dataset spans an extensive date range.

Postconditions:

- The user successfully retrieves, verifies, and audits the exact visual evidence and administrative history of any past detection event.
- The system maintains strict read-only access to these historical logs to preserve the legal integrity of the audit trail.

Use Case 10: Access Help Center

Actor: Operator, Administrator

Preconditions:

- The user is logged into the system.

Main Flow:

1. The user navigates to the "Help Center" module.
2. The system verifies the user's assigned role and dynamically renders the navigation menu. Operators are shown links to operational guides (e.g., "How to Verify an Alert"), while Administrators are provided with links to both operational and administrative guides (e.g., "How to Manage User Roles").
3. The user enters a specific question or keyword into the Help Center search bar or selects a predefined category tile.
4. The system filters the available documentation and displays a list of matching articles.
5. The user selects the relevant article.

6. The system displays the digital manual, allowing the user to review the instructional text and embedded images to resolve their issue.

Alternative Flows:

- 4a. Null Search Results: If the user enters a query that yields no matching documents, the system displays an empty-state graphic with the message: "No articles found for your search." To prevent a dead-end experience, the system automatically displays a list of Top Frequently Asked Questions beneath the error to guide the user to common, valid content.

Postconditions:

- The user successfully retrieves the necessary knowledge to operate the system.
- The system maintains strict visibility control, ensuring administrative guides remain hidden from standard Operators.

Use Case 11: Configure Alarm Settings

Actor: Operator, Administrator

Preconditions:

- The user is logged into the system.

Main Flow:

1. The user navigates to the "Alarm Settings" panel.
2. The system loads the page and populates all controls with the user's currently saved configuration.
3. The user selects a preferred alarm sound from the provided library of available audio options.
4. The system immediately plays a short preview of the selected sound at the currently configured volume level, allowing the user to evaluate the selection before saving.
5. The user adjusts the volume slider to their preferred level.

6. The system updates the audio preview in real time to reflect the adjusted volume.
7. The user enters a preferred snooze duration in seconds into the snooze configuration field.
8. The user clicks "Save Settings."
9. The system validates the input, commits the updated configuration to the database, and displays a success confirmation message.

Alternative Flows:

- 7a. Snooze Duration Below Minimum: If the user enters a value below 15 seconds, the system immediately flags the field with a validation error stating the minimum allowable duration and blocks the save action until a valid value is entered.
- 7b. Snooze Duration Above Maximum: If the user enters a value above 60 seconds, the system immediately flags the field with a validation error stating the maximum allowable duration and blocks the save action until a valid value is entered.
- 8a. No Changes Made: If the user clicks "Save Settings" without modifying any control, the system processes the request normally and confirms success without writing a redundant record to the database.

Postconditions:

- The user's alarm sound, volume level, and snooze duration are persisted in the database and immediately applied to all subsequent alerts within that user's active session.
- Upon the user's next login, the system restores the saved configuration, ensuring a consistent alert experience across sessions.

System Architecture and Design

The system is a localized edge-computing solution that automates continuous traffic camera monitoring. It ingests CCTV network streams and employs a custom-trained YOLO object detection model to identify vehicle-to-vehicle collisions in real time. Upon detection, the system captures visual evidence and transmits an immediate alert to a centralized command dashboard.

The Operations and Warning Division of the Lipa CDRMO serves as the primary user of the system. Automating the surveillance detection phase enables the system to overcome the limitations of manual monitoring of a large-scale citywide camera network, which has previously led to missed or delayed incident identification. The integration of AI-driven automation reduces dependence on delayed inter-agency endorsements and effectively addresses the emergency notification gap.

Client-Server Architecture Diagram

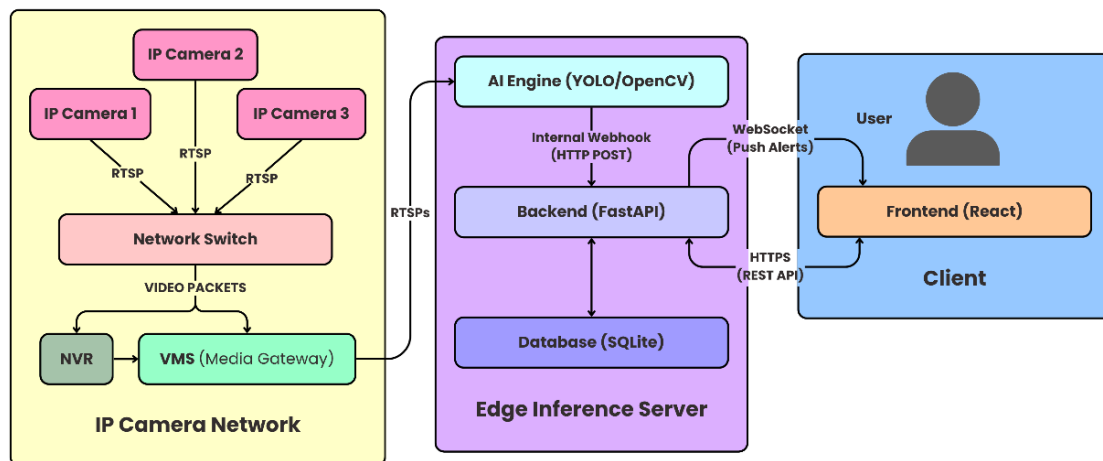


Figure 2.0. Client-Server Architecture

To meet the security, latency, and data privacy requirements of a government emergency command center, the system employed a localized client-server architecture

deployed on a single, on-premises edge inference server. This topology separated computationally intensive machine learning tasks from the client side.

All data ingestion, AI inference, database transactions, and Application Programming Interface (API) routing were performed within the Lipa CDRRMO's internal Local Area Network (LAN), eliminating reliance on remote cloud processing. The server handled raw video data and pushed JSON alerts to client browsers through WebSocket connections.

The Data Source (CCTV Infrastructure)

Although external to the software codebase, the agency's physical IP cameras formed the foundational data layer. These cameras streamed live video over the local network via RTSP, providing a continuous input feed to the system's AI layer.

The AI Engine

This component used OpenCV to ingest and decode RTSP streams. The decoded frames were processed by the Ultralytics YOLO framework for real-time inference. Upon collision detection, the module captured a bounding-box snapshot and sent a secure internal HTTP (Hypertext Transfer Protocol) webhook to the backend to register the event.

The Backend

Developed with FastAPI and deployed on an ASGI server (Uvicorn), it managed asynchronous routing and business logic. The backend received alerts from the AI engine, enforced RBAC through JWT validation, managed hardware

telemetry polling, and maintained a persistent WebSocket manager to deliver live data to frontend clients.

The Database

SQLite was selected to minimize network latency and eliminate the overhead of a standalone database engine on the edge server. Configured with Write-Ahead Logging (WAL) to support concurrent reads and writes, it securely stored user credentials, historical detection logs, and hardware telemetry. All database interactions were validated and sanitized by the SQLAlchemy Object-Relational Mapper (ORM) to maintain structural integrity.

The Frontend

The client side was a dynamic SPA developed with React. Used in standard web browsers at the command center workstations, it provided the user interface. Zustand was used for rapid state management of incoming WebSocket payloads, while TanStack Query handled RESTful (Representational State Transfer) data caching.

Swimlane Diagram

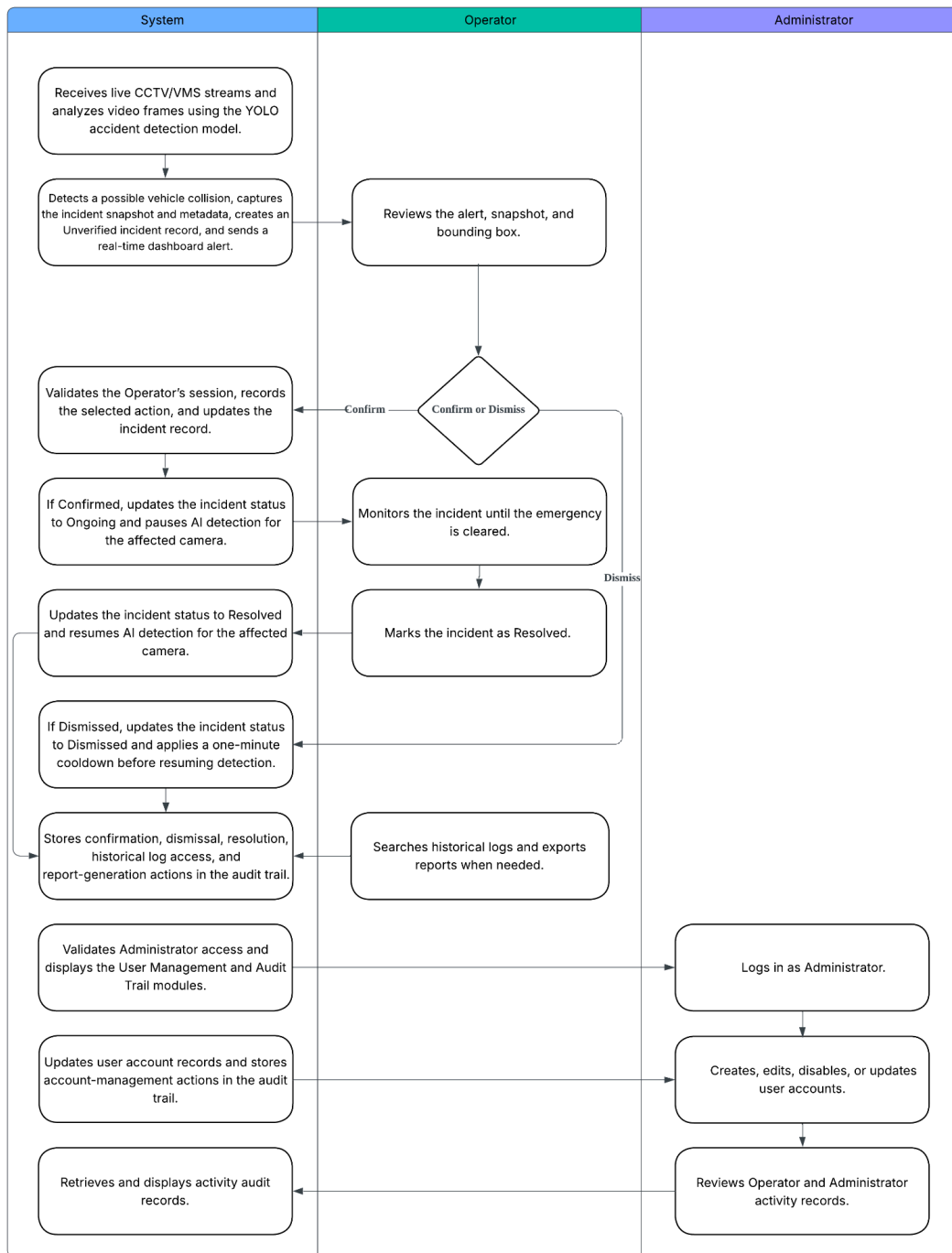


Figure X.X. Swimlane Diagram

The swimlane diagram illustrates how the proposed system distributes tasks among the System, Operator, and Administrator during accident detection and verification. The System lane covers the automated flow, beginning with the

reception of CCTV/VMS streams, AI-based frame analysis, possible collision detection, incident record creation, and dashboard alert delivery. The Operator lane represents the Human-in-the-Loop portion of the workflow, where the dispatcher reviews the alert snapshot, decides whether to confirm or dismiss the detection, monitors confirmed incidents, marks cleared incidents as resolved, and generates historical reports when needed. The Administrator lane shows the restricted account-management responsibilities assigned to authorized supervisory users.

This diagram is included to clarify responsibility flow rather than internal software structure. Unlike the system architecture diagram, which identifies the major technical components, the swimlane focuses on who performs each action and how the workflow moves between automated processing, operator decision-making, and administrative control. This supports the operational context of the Lipa CDRRMO, where AI detection assists the command center but final incident verification remains under human supervision.

Use Case Diagram

operational structure is designed to maximize efficiency, mitigate operator cognitive load, and accelerate the Human-in-the-Loop (HITL) emergency response.

The system's operational hierarchy consists of three primary actors. The first is the Operator, also known as the Dispatcher, who serves as the frontline user responsible for managing the command center. This position includes comprehensive control over operational features such as monitoring AI alerts, configuring cameras, managing alarm settings, generating analytical reports, and tracking both hardware health and AI performance. The second actor is the Administrator, sometimes referred to as IT or Supervisor. The Administrator possesses all operational capabilities of the Operator and holds exclusive authority to manage the user directory and account access. The third actor, the CCTV System, functions as the city's enterprise Video Management System (VMS) proxy and provides an automated, continuous data pipeline to the edge server.

Secure Access and Session Routing

Every operational shift begins with the Login process. To ensure the integrity of the command center, the system must Determine User Role to route the user to the appropriate dashboard and Validate Credentials against the encrypted database. If authentication fails, the system executes a Display Login Error message. Once successfully authenticated, all users can manage their own settings, such as Edit Own Profile or Change Own Password.

The Human-in-the-Loop Resolution Pipeline

The system's primary function is its automated emergency detection pipeline. The external CCTV System continuously executes the Stream Camera Feed function, allowing the AI edge server to autonomously Detect Accident in the background.

When an accident is detected, the system requires human intervention through the Verify Accident Alert workflow. To mitigate operator cognitive load during this verification phase, the detection workflow is extended by the Snooze Auditory Alarm function. This optional extension allows the dispatcher to temporarily suppress the system's auditory alarm, providing a silent operational window to evaluate the accident snapshot without distraction. Depending on the dispatcher's evaluation of the accident, the workflow proceeds in one of two ways. If the dispatcher confirms the accident as a true positive, they validate the collision and instruct the system to Set Ongoing Accident Status. This action pauses the AI inference on that specific camera, preventing redundant alerts while emergency services are dispatched. Alternatively, if the dispatcher dismisses the alert as a false positive, identifying an environmental anomaly such as heavy traffic, they instruct the system to set a one-minute cooldown timer. This temporarily silences the AI for that camera.

By requiring the dispatcher to trigger either the Dismiss Accident or Resolve Accident function, the system ensures that every AI detection is permanently recorded with a definitive human audit trail.

Camera Configuration Management

To enable the command center to adapt to network changes or new camera installations, standard Operators are empowered to Manage Cameras. Through the operational dashboard, dispatchers can perform administrative tasks such as Add New Camera, Edit Camera Configuration, View Camera Status, or Delete Camera Feed. This operational structure allows the dispatch team to directly maintain the system's video input sources. In addition to video management, Operators are granted the ability to Configure Alarm Settings. This function allows dispatchers to customize auditory alert tones, adjust system volume levels, and define the specific temporal duration for the

snooze timer, ensuring the dashboard is continuously optimized for the team's acoustic environment.

Command Center Telemetry and Reporting

Because dispatchers oversee the complete lifecycle of an incident, they are equipped to manage system analytics. To support inter-agency reporting and related tasks, Operators can Analyze Accident Analytics to identify dangerous intersections or Query Historical Logs for specific incident audits. When physical records are required by the CDRRMO, Operators use the Export Reports function to generate formatted reports. Additionally, to maintain system stability during a crisis, the dispatch team can actively Monitor System Health to track the physical limits of the edge server's CPU, GPU, and RAM usage, as well as Track AI Performance metrics such as inference latency and model efficiency.

User Account Administration

The Manage User Accounts workflow is exclusively reserved for Administrators. When accessing this secured directory, Administrators are provided with a comprehensive suite of access-control tools necessary to maintain the dispatch team's security, including the ability to Add User Account, Edit User Account, Delete User Account, and forcefully Change User Password.

Data Flow Diagram (DFD)

Context Level

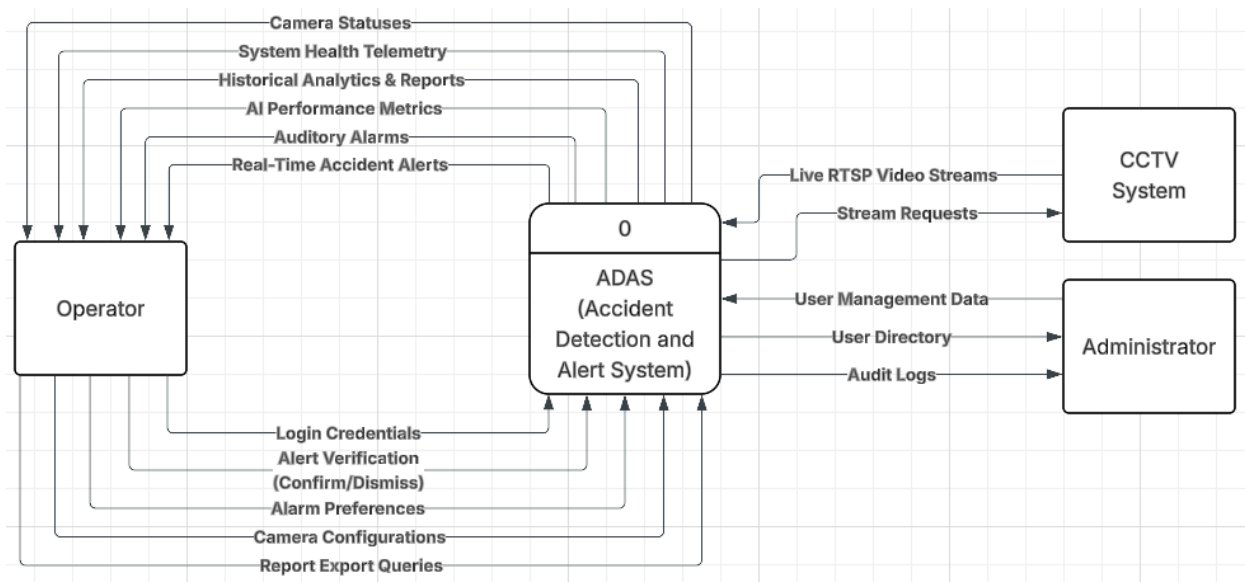


Figure 5.0 Context Level DFD

The Context Level (DFD establishes the operational boundaries of the system by representing the entire platform as a single process interacting with three distinct external entities. The primary data source is the CCTV System, which maintains a continuous input-output relationship with the system by providing live video streams in response to specific stream requests. The HITL architecture is primarily facilitated through the Operator entity, who supplies critical inputs such as login credentials, verification decisions for AI-flagged incidents, camera configuration data, report export filters, and alarm preferences. In return, the system provides the Operator with real-time accident alerts alongside auditory alarms, historical analytics, system health telemetry, and AI performance metrics.

The Administrator entity serves as the secondary human actor, focusing on higher-level system governance and security. While the Administrator inherits the data outflows provided to the Operator, the diagram highlights the unique administrative data loop where the Administrator provides user management data and receives comprehensive user directories and audit logs. This centralized structure ensures that all

localized edge computing tasks, including AI inference and database management, remain isolated within the system process while maintaining clear, secure communication channels with the Lipa CDRRMO's infrastructure and personnel.

Level 1

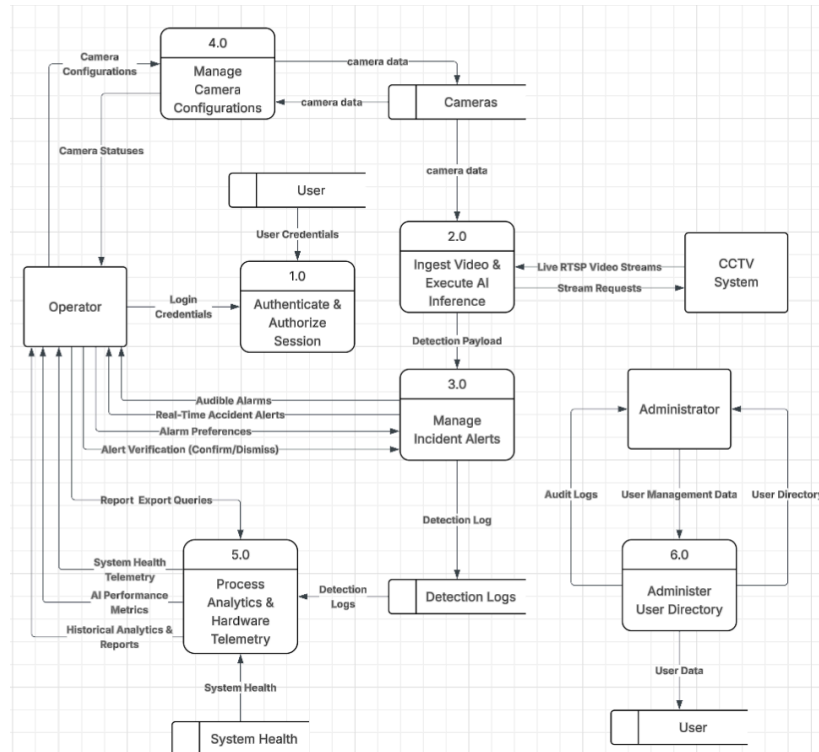


Figure 5.1 Level 1 DFD

The Level 1 Data Flow Diagram illustrates the internal decomposition of the system, expanding the single process from the context level into six primary sub-processes supported by four localized data stores. This level of detail establishes the functional relationships between the user roles, the AI detection engine, and the underlying database architecture. The Authenticate and Authorize Session (1.0) process serves as the entry point for both Operators and Administrators, validating credentials against the User Data Store (D1) to establish role-based access control (RBAC).

The core technical pipeline begins with Ingest Video and Execute AI Inference (2.0), which retrieves live RTSP streams from the external CCTV infrastructure and references the Camera Data Store (D2) for configuration metadata. When a collision is identified, the payload is transmitted to the Manage Incident Alerts (3.0) process. This process serves as the hub for the Human-in-the-Loop (HITL) workflow, recording initial detections in the Detection Log Data Store (D3) and pushing real-time WebSocket alerts and auditory alarms to the Operator. The Operator's subsequent verification decisions are routed back through this process to update the incident status in the database.

Parallel to the detection pipeline, Manage Camera Configurations (4.0) allows Operators to update the Camera Data Store (D2), which dynamically informs the stream requests sent to the VMS. For long-term oversight, Process Analytics and Hardware Telemetry (5.0) pulls historical records from the Detection Log (D3) and hardware metrics from the System Health Data Store (D4) to generate the visualizations and reports requested by the Operator. Finally, the Administer User Directory (6.0) process remains isolated for Administrator use, facilitating the management of user account profiles and security credentials within the User Data Store (D1). Together, these processes ensure a balanced flow of information that maintains both real-time responsiveness and administrative integrity.

Entity Relationship Diagram (ERD)

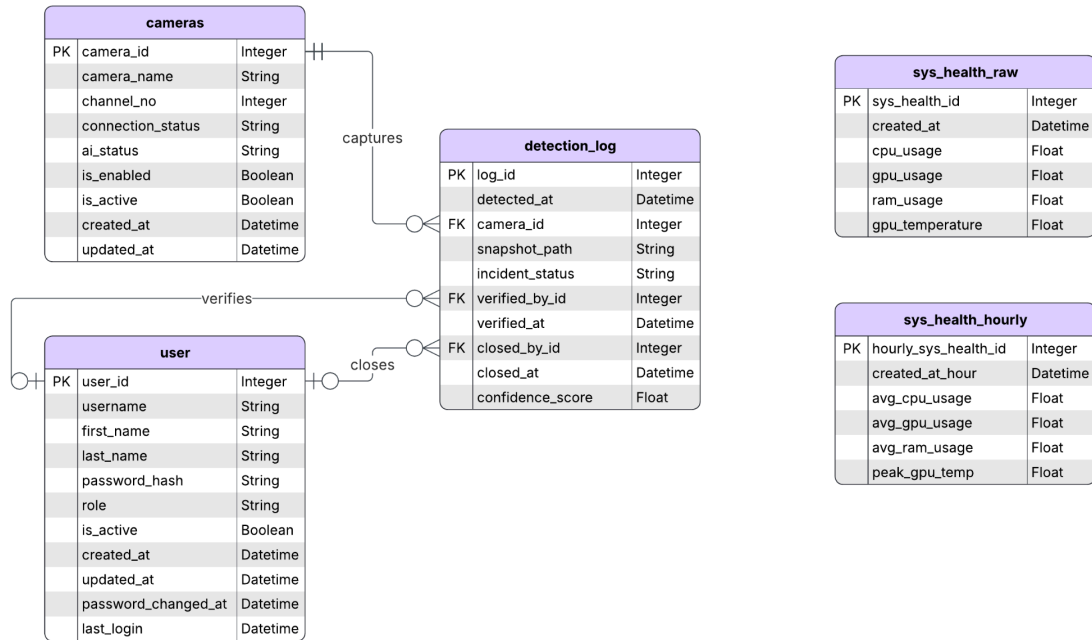


Figure 6.0. Entity-Relationship Diagram of the Database Architecture

The database's operational workflow was structured around three primary entities: user, camera, and detection_log. Each camera served as the source for multiple accident logs, establishing a mandatory one-to-many relationship from camera to detection_log, since a detection log could not exist without a hardware source. As incident records were created before operator involvement, the verification relationship (verified_by_id) between the user and detection_log was optional. This field was populated only when an operator interacted with the HITL dashboard to confirm or dismiss the alert, ensuring that verified accidents were directly linked to the responsible operator. If a collision was confirmed (Ongoing), the accident remained open and unresolved. The closure relationship (closed_by_id) was therefore also optional to populate the field only when the scene was officially cleared or dismissed.

The sys_health_raw and sys_health_hourly tables were intentionally separated from operational relationships. Linking them to the user or camera tables via foreign keys would have required the database engine to perform unnecessary structural validations on each insert.

The relational database was extensively optimized and strictly adhered to 3NF to eliminate data redundancy, prevent update anomalies, and maximize query efficiency. First Normal Form (1NF) was achieved by assigning explicit, auto-incrementing integer primary keys (e.g., user_id, camera_id, etc.) to each table, ensuring row uniqueness and atomic attributes without repeating groups or comma-separated lists. The use of single-column, synthetic primary keys rather than composite keys inherently satisfied Second Normal Form (2NF), as every non-key attribute was fully functionally dependent on the entire primary key. The schema achieved 3NF by eliminating transitive dependencies. Only foreign keys were stored. This design ensured that updates to a camera's name or corrections to a user's name were made in a single location, and the accident history automatically reflected these changes through SQL JOINS, thereby preventing data inconsistency.

Although the core system enforced strict 3NF, the sys_health_hourly table introduced controlled denormalization. Rather than requiring the edge server to compute computationally intensive aggregate functions across numerous raw telemetry rows each time an administrator accessed the analytics dashboard, the system precalculated and stored these values at the start of each hour. This data warehousing strategy prioritized read performance and CPU efficiency over strict normalization for historical metrics.

Data Dictionary

Table 4.0. User

Field Name	Data Type	Description	Constraints
user_id	Integer	Unique identifier for the user. Auto-generated.	Primary Key, Auto-increment
username	String	The login credential used to access the dashboard.	Unique, Not Null
first_name	String	The user's first name	Not Null
last_name	String	The user's surname.	Not Null
password_hash	String	The bcrypt-hashed password.	Not Null
role	String	Defines system access level (Admin or Operator).	Not Null
is_active	Boolean	Used for 'soft deletes' so records are not permanently wiped	Not Null, Default: True
created_at	Datetime	Timestamp of when the account was created.	Not Null, Default: Current Time
updated_at	Datetime	Timestamp of the most recent profile edit (Name, Username, Role).	Not Null, Default: Current Time, Auto-updates
password_changed_at	Datetime	Timestamp of the most recent password change.	Nullable
last_login	Datetime	Timestamp of the user's most recent successful login.	Nullable

Stored account credentials, biographical data, and RBAC levels for system operators and administrators, as well as critical security audit timestamps for tracking access and password changes.

Table 5.0. Camera

Field Name	Data Type	Description	Constraints
camera_id	Integer	Unique identifier for the camera. Auto-generated.	Primary Key, Auto-increment
camera_name	String	Display name for the dashboard (e.g., "Crossing - Banaybanay").	Unique, Not Null
channel_id	Integer	The VMS Channel ID. Used dynamically by the backend,	Unique, Not Null

		together with global environment credentials, to request a specific camera feed.	
connection_status	String	Current RTSP stream connection state (Connected, Disconnected, Reconnecting, Unresponsive).	Not Null, Default: 'Disconnected'
ai_status	String	Current YOLO thread state (Active, Inactive, Paused, Unresponsive).	Not Null, Default: 'Inactive'
is_enabled	Boolean	Toggle to start or stop RTSP stream ingestion intentionally.	Not Null, Default: True
is_active	Boolean	Used for "soft deletes" to preserve historical accident links.	Not Null, Default: True
created_at	Datetime	Timestamp of when the camera was added to the system.	Not Null, Default: Current Time
updated_at	Datetime	Timestamp of the most recent edit to this specific record.	Not Null, Default: Current Time, Auto-updates

Managed the configuration and operational status of deployed IP cameras, including their physical network connections, AI processing status at the software level, and the state of the RTSP stream ingestion.

Table 6.0. Detection Log

Field Name	Data Type	Description	Constraints
log_id	Integer	Unique identifier for the accident. Auto-generated.	Primary Key, Auto-increment
detected_at	Datetime	The exact timestamp at which the AI detected the collision.	Not Null
camera_id	Integer	Links the accident to the camera that recorded it.	Foreign Key (Camera.camera_id), Not Null
snapshot_path	String	The local file path to the saved image.	Not Null
detection_status	String	The verification state (Unverified,	Not Null, Default:

		Ongoing, Dismissed, Resolved).	'Unverified'
verified_by_id	Integer	Links the accident to the user who clicked Confirm/Dismiss on the initial Accident Alert modal.	Foreign Key (User.user_id), Nullable
verified_at	Datetime	The exact time the operator clicked Confirm/Dismiss on the initial Accident Alert modal.	Nullable
closed_by_id	Integer	Links the accident to the user who clicked Resolved/Dismiss after reopening an ongoing incident.	Foreign Key (User.user_id), Nullable
closed_at	Datetime	The exact time the operator clicked Resolved/Dismiss after reopening an ongoing incident.	Nullable
confidence_score	Float	The probability score (e.g., 0.88 for 88%) outputted by the YOLO model indicating its certainty of the vehicle collision.	Not Null

Served as the primary forensic archival table, recording AI-detected vehicular collisions, model confidence scores, and the complete HITL chronological audit trail for accident verification and scene clearance.

Table 7.0. System Health Raw

Field Name	Data Type	Description	Constraints
sys_health_id	Integer	Unique identifier for the log entry. Auto-generated.	Primary Key, Auto-increment
created_at	Datetime	The exact time the background monitor polled the hardware.	Not Null
cpu_usage	Float	The percentage of the edge server's total CPU load.	Not Null
gpu_usage	Float	The percentage of the edge server's total GPU load.	Not Null
ram_usage	Float	The percentage of the edge server's active RAM usage.	Not Null
gpu_temperature	Float	The thermal reading of the GPU	Not Null

		(in Celsius).	
--	--	---------------	--

Logged high-resolution, hardware performance metrics of the edge server, polled at five-minute intervals to monitor immediate operational stability, and is pruned after 48 hours.

Table 8.0. System Health Hourly

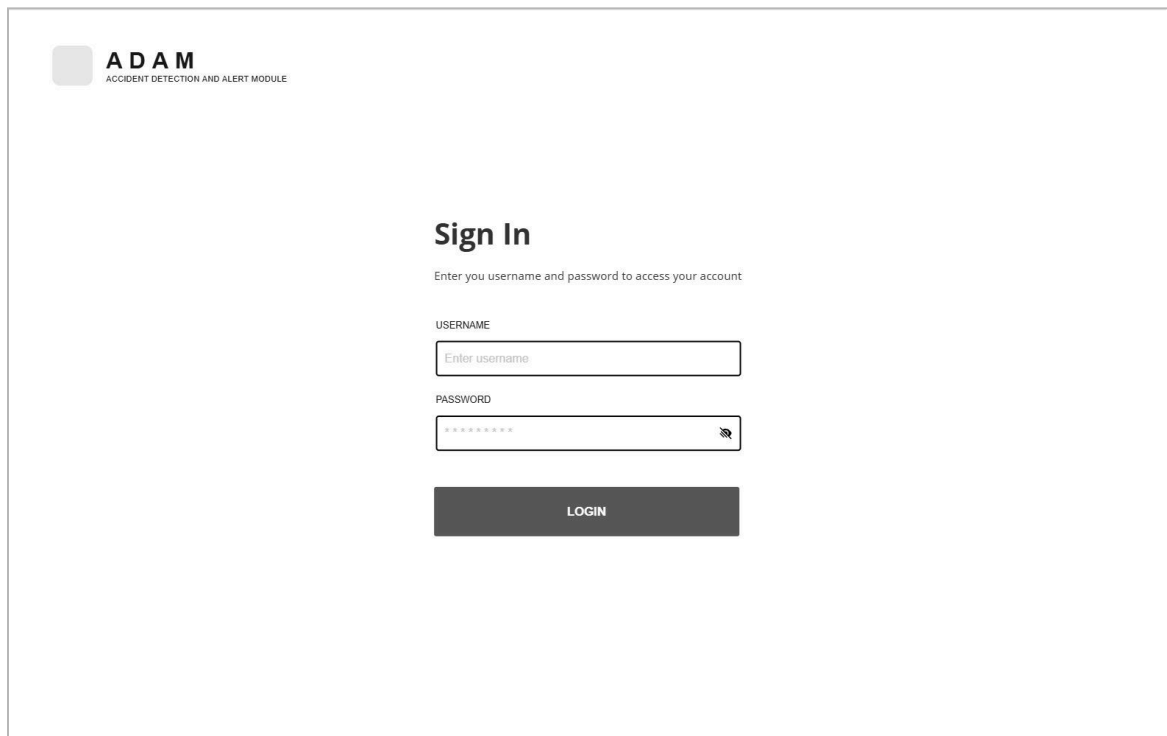
Field Name	Data Type	Description	Constraints
hourly_sys_health_id	Integer	Unique identifier for the hourly summary. Auto-generated.	Primary Key, Auto-increment
created_at_hour	Datetime	The specific hour block this row represents (e.g., 2026-04-13 14:00).	Not Null, Unique
avg_cpu_usage	Float	The calculated average CPU usage for that hour.	Not Null
avg_gpu_usage	Float	The calculated average GPU usage for that hour.	Not Null
avg_ram_usage	Float	The calculated average RAM usage for that hour.	Not Null
peak_gpu_temp	Float	The maximum GPU temperature reached during that hour.	Not Null

Stored aggregated, low-resolution hardware performance trends calculated hourly, retained for a rolling 30-day period to support long-term system stability analysis and to populate the analytics dashboard efficiently.

Wireframes

The wireframes below depict the interface layouts for each major system view. Given the system's deployment in emergency response contexts, the design philosophy

prioritizes simplicity, functional reliability, and immediate accessibility rather than aesthetic embellishment.



The image shows a wireframe of a login page for a system called ADAM (Accident Detection and Alert Module). In the top-left corner, there is a logo consisting of a grey square followed by the text "ADAM" in a bold, sans-serif font, with "ACCIDENT DETECTION AND ALERT MODULE" in a smaller font below it. The main content is centered and starts with the heading "Sign In" in a bold font. Below this heading is a subtext: "Enter you username and password to access your account". There are two input fields: the first is labeled "USERNAME" and contains the placeholder text "Enter username"; the second is labeled "PASSWORD" and contains a series of dots, with a small eye icon to its right indicating a password visibility toggle. Below these fields is a large, dark grey rectangular button with the word "LOGIN" in white, uppercase letters.

Figure 7.1. Login Page

This figure presents the wireframe of the authentication interface, with the system logo in the upper-left corner and the primary authentication form centered within the viewport.

The form includes clearly labeled input fields for username and password, a toggle for password visibility, and a prominent submit button. The login button features a large target area to reduce interaction time and support rapid, error-free access. Maximized white space and the removal of unnecessary visual elements minimize cognitive distraction and facilitate efficient access to the operational dashboard.

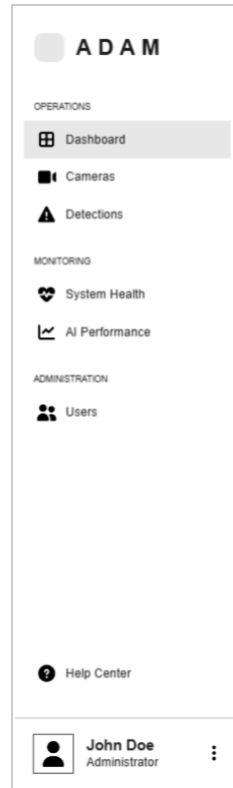


Figure 7.2. Sidebar Panel

The fixed left sidebar navigation panel remains visible at all times, ensuring immediate, unobstructed access to system modules during emergency alerts and eliminating the need to toggle menus.

The layout features the system logo at the top, with a persistent Help Center link and user profile block at the bottom. The profile block displays the active user's name and assigned role, accompanied by an ellipsis icon for contextual account management actions. To reduce cognitive overload, the navigation hierarchy is divided into small, logical groups: Operations, Monitoring, and Administration. Access to the Administration section, which contains the Users module, is controlled by Role-Based Access Control and is only available to Administrator accounts. Each navigation link is paired with a distinct icon to support rapid visual identification.

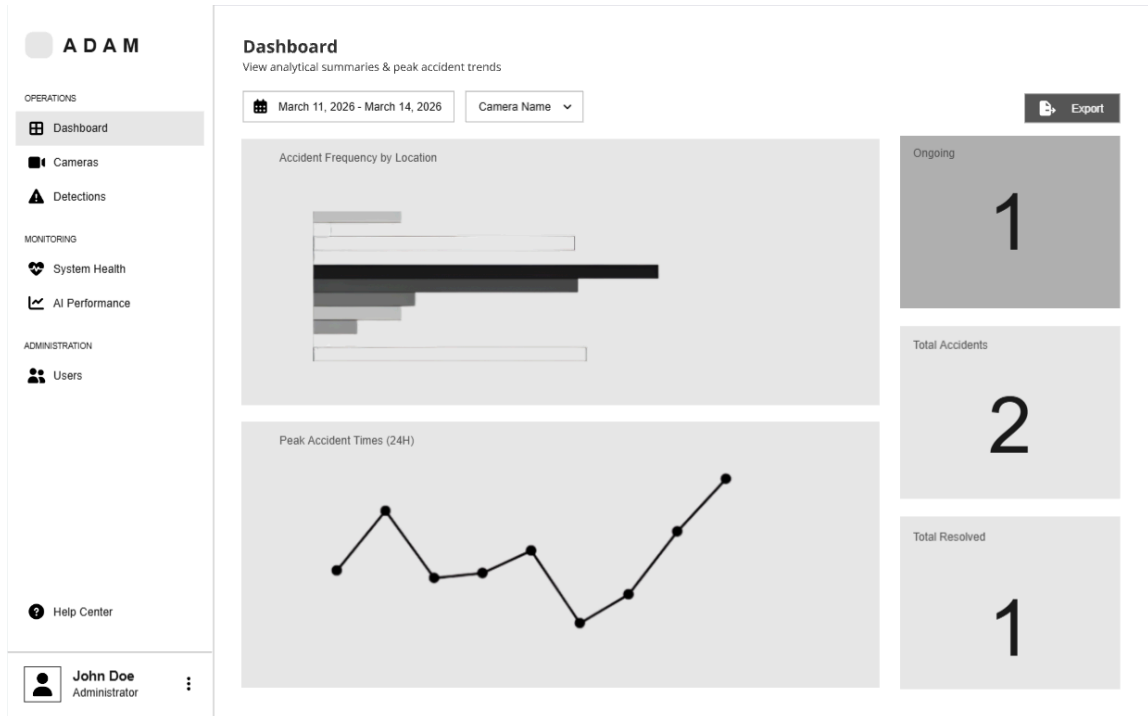


Figure 7.3. Dashboard

The primary Dashboard layout is accessed via the "Dashboard" module in the left navigation sidebar. As the central analytical interface, the viewing area employs a consistent typographic hierarchy, featuring a prominent page title and a descriptive subtitle to orient users and provide immediate functional context.

Below the header, discrete filtering controls, including a date range selector and a camera dropdown, are prominently displayed rather than hidden within a toggle menu. This approach minimizes the number of interaction steps for the users. The "Export" button, located at the top right, serves as the primary call-to-action and is distinguished by its larger size, contrasting background, and icon.

The core data presentation consists of graphical reports and numerical summaries. A horizontal bar graph displays "Accident Frequency by Location," while a time-series line graph tracks "Peak Accident Times (24H)." Adjacent to these charts, three KPI cards summarize critical operational metrics. The "Ongoing" incidents card is

emphasized with a darker visual weight to draw user attention, as unresolved emergencies require monitoring and immediate human verification to resume the AI detection for affected camera feeds. The remaining cards track cumulative accident and resolution volumes.

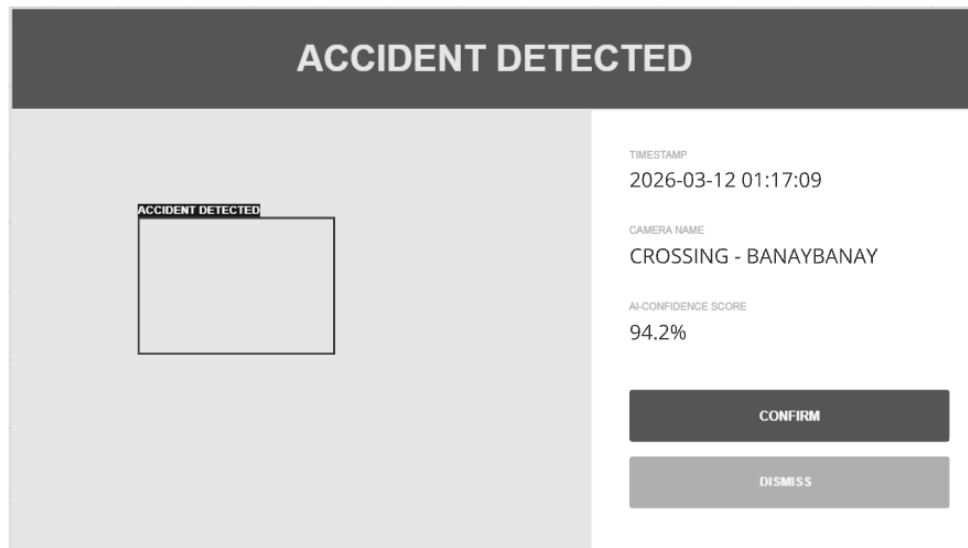


Figure 7.4. Accident Alert Modal

The accident alert modal is designed to ensure rapid human validation of AI-flagged accidents. It appears as a system-wide, high-visibility overlay immediately upon AI detection of a collision, interrupting the operator's current activity to demand attention.

The modal is horizontally divided for efficient situational assessment: the left panel displays a captured incident snapshot with an AI-drawn bounding box, while the right panel presents critical incident telemetry, including Timestamp, Camera Name, and AI-Confidence Score, using enlarged typography for readability. Below the metadata, two essential action buttons ("Confirm" and "Dismiss") feature large target areas for rapid interaction, as both actions are critical to workflow management. The interface omits a standard window-close ('X') control to enforce the HITL verification workflow, requiring

operators to evaluate the evidence and record a definitive decision, thus preventing passive or accidental dismissal of alerts.

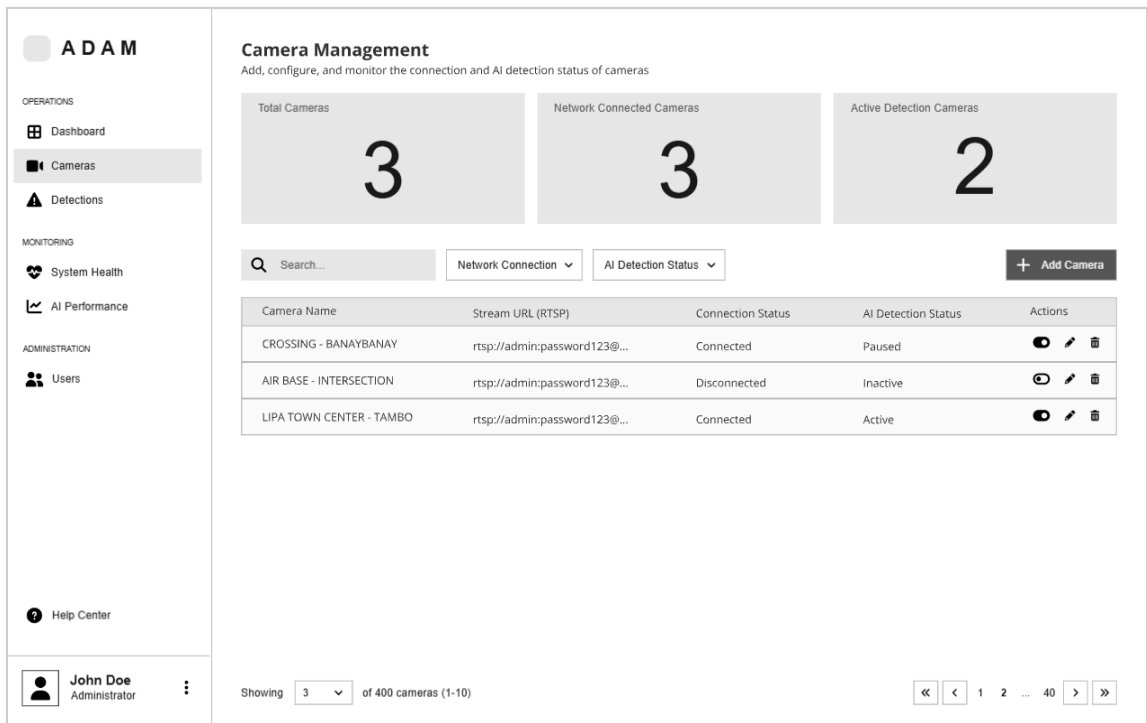


Figure 7.5. Camera Management Page

The Camera Management interface is accessible through the "Cameras" module in the left navigation sidebar. The interface is designed to enable precise configuration and real-time monitoring of the deployed CCTV network and its associated AI inference processes.

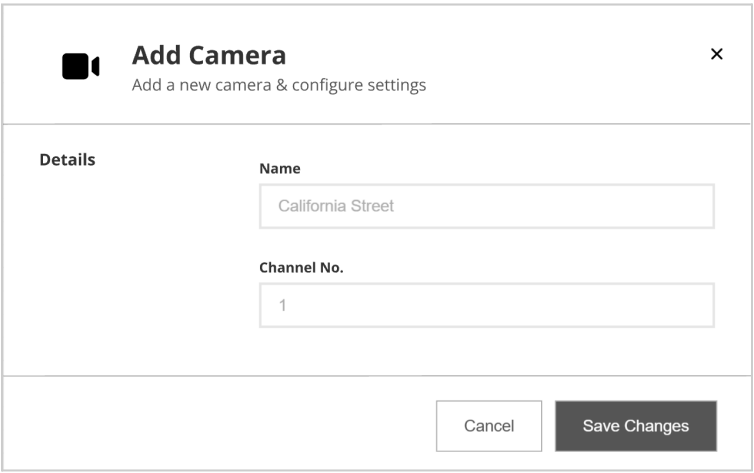
At the top of the layout, a clear typographic hierarchy is established: the primary page title appears in a large font, and the subtitle offers immediate functional context to orient users. Directly below the header, three prominent numerical metrics provide administrators and operators with high-level situational awareness of the total hardware count, active network connections, and ongoing AI detection statuses.

A dedicated query toolbar is positioned directly beneath these KPIs, integrating a dynamic search input and dropdown filters for "Network Connection" and "AI Detection

Status" to enable rapid identification of specific camera feeds. The primary call-to-action, the "Add Camera" button, is strategically anchored at the top right and distinguished by a contrasting background and clear icon.

The central component is a structured data table listing individual camera configurations and their real-time operational states. The interactive 'Actions' column is organized by usage frequency and operational risk: routine controls, such as the active state toggle and edit functions, precede the delete action, which is isolated at the far right to reduce the risk of accidental data loss.

A standardized pagination module at the bottom of the viewport uses row-count selectors and directional navigation to maintain a consistent data traversal pattern across all tabular interfaces.



The image shows a modal window titled "Add Camera" with a close button (X) in the top right corner. Below the title is a subtitle "Add a new camera & configure settings". The main content area is labeled "Details" and contains two input fields: "Name" with the value "California Street" and "Channel No." with the value "1". At the bottom right of the modal are two buttons: "Cancel" and "Save Changes".

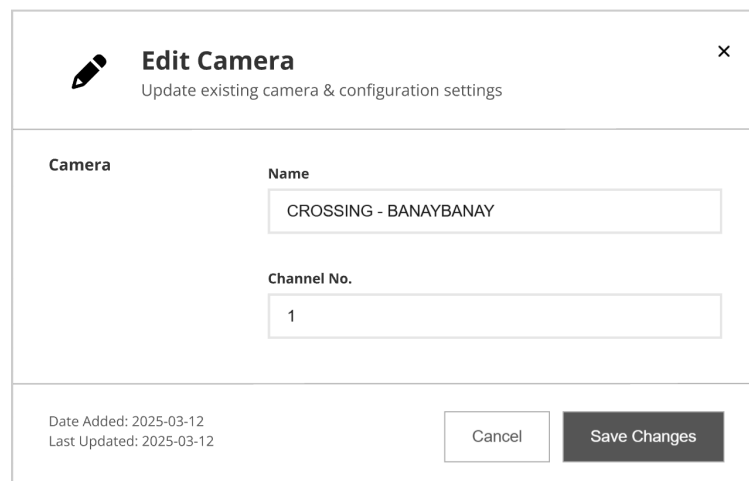
Figure 7.6. Add Camera Form Modal

The Add Camera Form Modal enables precise input of CCTV network configurations and is accessed by clicking the "Add Camera" button on the Camera Management Page (Figure 3.4).

The modal header features a standardized layout with an identifying icon, primary title, contextual subtitle, and a standard window-close ('X') control in the upper right, aligning with user interface expectations.

Each input field includes explicit labels and contextual placeholder examples to guide operators and minimize formatting errors before the system adds the channel no. to the functional URL.

The footer contains two action controls: the "Save Changes" button uses a high-contrast background to indicate the primary action, while the secondary "Cancel" button is visually deprioritized with a low-contrast, outlined design.



The image shows a modal window titled "Edit Camera" with a pencil icon and a close button (X). Below the title is a subtitle: "Update existing camera & configuration settings". The main content area is divided into two sections. The first section, labeled "Camera", contains a "Name" input field with the text "CROSSING - BANAYBANAY". The second section, labeled "Channel No.", contains a "Channel No." input field with the text "1". At the bottom of the modal, there is a footer area containing the text "Date Added: 2025-03-12" and "Last Updated: 2025-03-12" on the left, and two buttons on the right: "Cancel" (outlined) and "Save Changes" (solid dark background with white text).

Figure 7.7. Edit Camera Form Modal

This structurally mirrors the Add Camera Form Modal (Figure 3.5) to ensure visual consistency. Access is provided via the edit icon (pencil) in the Actions column of Figure 3.4.

To facilitate efficient updates, the input fields are dynamically pre-populated with the camera's current Name and Channel ID, allowing operators to make targeted modifications efficiently.

A significant architectural enhancement is the addition of read-only metadata at the bottom left, where the interface displays the "Date Added" and "Last Updated" timestamps to support administrative auditability and record tracking. The footer retains the standardized action controls, with high-contrast styling on the "Save Changes" button to clearly indicate the primary action.

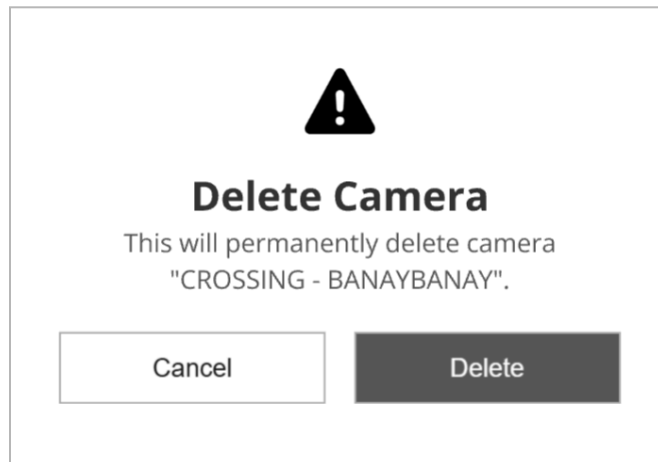


Figure 7.8. Delete Camera Confirmation Modal

This critical fail-safe interface intercepts destructive system actions, triggered when an operator or administrator selects the delete button in the Actions column of Figure 3.4.

The centered layout features a prominent warning icon at the top to emphasize the severity of the impending action. Below the primary title, descriptive text explicitly identifies the hardware entity to be deleted ("CROSSING - BANAYBANAY") and outlines the specific consequence, avoiding generic confirmation dialogue.

To reduce the risk of accidental data loss, the modal employs explicit action labeling in its dual-button footer. The primary destructive control, labeled "Delete," uses a high-contrast warning color, while the secondary safe-exit control, labeled "Cancel," employs a low-contrast outlined design to provide a clear, non-destructive exit option.

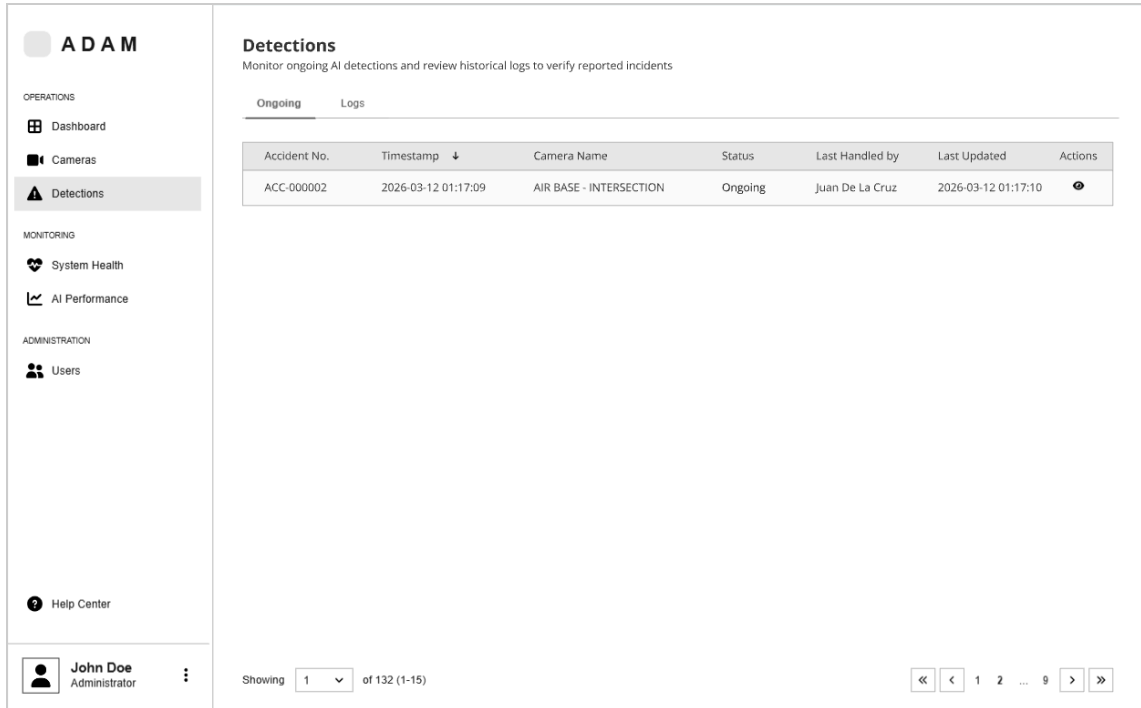


Figure 7.9. Detections Ongoing Tab

This Detections interface focuses on the Ongoing accidents view, which is accessed by selecting the "Detections" module from the left navigation sidebar. The primary purpose is to enable operators and administrators to continuously monitor actively verified collisions requiring scene clearance and final resolution.

Aligned with the system's design language, the layout features a prominent page title and a descriptive subtitle for immediate functional context. Below the header, a tabbed navigation structure separates current emergencies ("Ongoing") from historical data ("Logs"). The "Ongoing" tab is set as the default view, ensuring users see critical, unresolved accidents when they access the module.

The central component is a structured data table containing only accident records with an "Ongoing" status, detailing metadata such as accident number, timestamp, location, and the specific personnel handling the event. The interactive Actions column

includes a distinct "View" icon, enabling operators to open the detailed incident modal to review visual evidence or update operational status.

The interface also incorporates the standardized pagination module at the bottom to facilitate efficient navigation through active incident records.

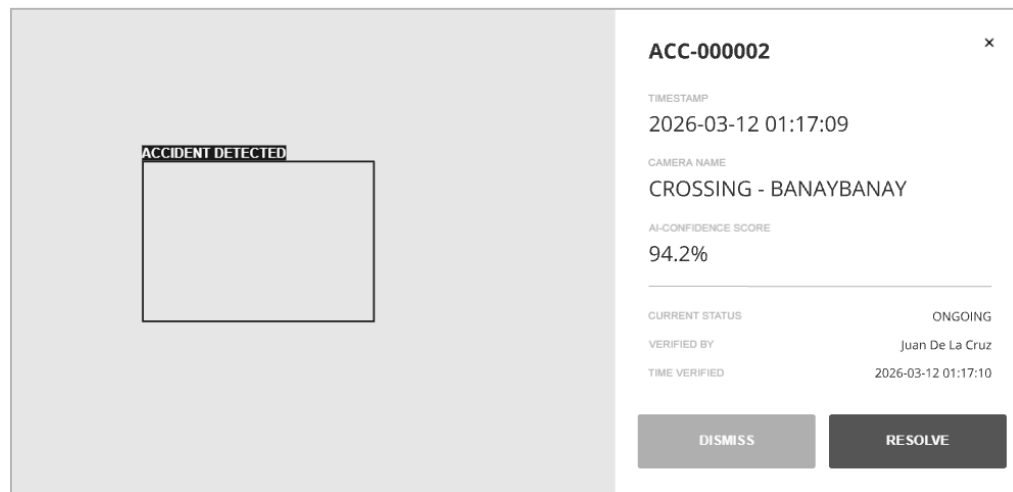


Figure 7.10. Ongoing Accident Details Modal

This Ongoing Accident Details Modal facilitates the database via a scheduled cron job. Because the database operates in Write-Ahead Logging (WAL) mode to support concurrent read/write operations, standard file copying commands pose a severe risk of data fragmentation. To safely capture the data state without interrupting active AI inference or triggering database locks, the protocol strictly utilizes the native sqlite3 .backup command to generate structurally sound, point-in-time database snapshots. While this process is automated, the Administrator can manually trigger immediate backups prior to configuration changes.

- Tier 2 (Automated Air-Gapped Archival with Manual Oversight): To protect against catastrophic physical server failure, local backups are systematically offloaded. The system securely transfers the database snapshots, alongside compressed visual incident evidence and the custom-trained YOLO model weights, to an air-gapped Network Attached

Storage (NAS) managed by the Lipa CDRMO. To minimize human error, this weekly transfer is automated via background scripts. However, the operational oversight of this is strictly manual—including the verification of data integrity and the physical security of the external storage—is the explicit responsibility of the Command Center Supervisor (Administrator).

Restoring a database within an active edge environment presents a critical concurrency challenge: a REST API cannot safely overwrite an active database file while the FastAPI backend and AI worker threads are continuously executing read and write operations. Therefore, the restoration process is designed to be manually initiated but automatically executed.

The Administrator, acting as the system Administrator, is the sole personnel authorized to restore the system. When a critical failure occurs, the Administrator manually selects a specific restore point via the web dashboard.

Once initiated, the system takes over via an automated "Flag and Restart" architecture:

1. The backend automatically generates a secure "restore flag" (a temporary system file) containing the target backup identifier.
2. The system issues an automated reboot command to the application services.
3. During the reboot sequence, an automated Linux systemd pre-start script intercepts the boot process while the applications are safely offline.
4. This script automatically overwrites the primary database file and critically deletes the lingering -wal and -shm sidecar files to prevent corruption.

- The script clears the flag, automatically completing the boot sequence and hydrating the dashboard for the Supervisor with the restored historical data.

Deployment Architecture

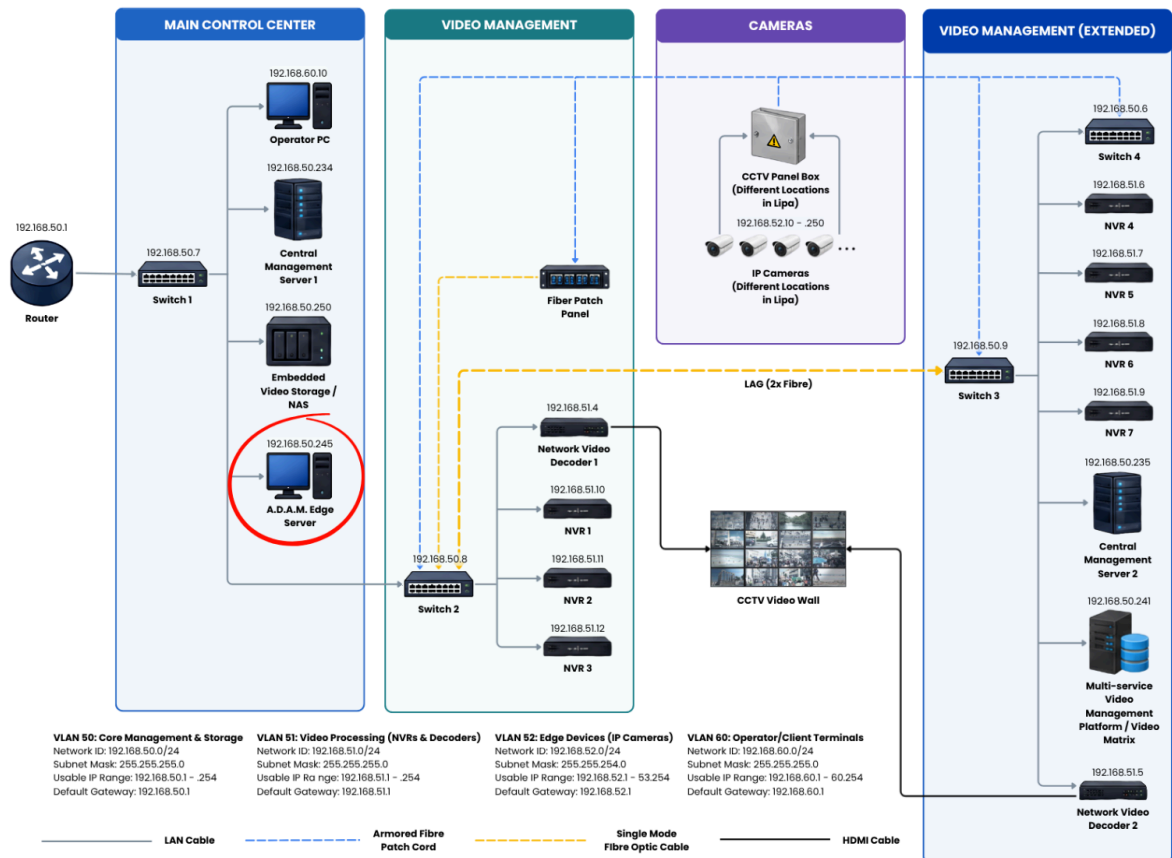


Figure 8.0. Deployment Architecture Diagram

To optimize data flow and isolate computationally intensive AI operations from mission-critical video archiving, the infrastructure employs IEEE 802.1Q Virtual Local Area Network (VLAN) segmentation. The logical network topology is divided into four distinct broadcast domains to prevent bandwidth bottlenecks:

- VLAN 50 (Core Management): Houses the edge inference server, central management servers, and NAS arrays. This segment is strictly dedicated to AI processing, administrative routing, and database transactions.
- VLAN 51 (Video Archival): Isolates the Network Video Recorders (NVRs) and network video decoders, ensuring continuous, high-definition recording traffic does not congest the operational management layer.
- VLAN 52 (Edge Surveillance): Encompasses the localized IP camera network, securing the raw video packet transmission layer.
- VLAN 60 (Operations): Isolates the standard dispatch workstations utilized by command center personnel to access the web dashboard.

Cross-VLAN communication is executed via a Router-on-a-Stick topology. The core router utilizes trunking encapsulation and virtual sub-interfaces to securely route traffic between these isolated networks. This configuration enables the edge server (VLAN 50) to securely transmit real-time WebSocket alerts and REST API payloads to the operator terminals (VLAN 60) without exposing the overarching command center network to unnecessary broadcast traffic.

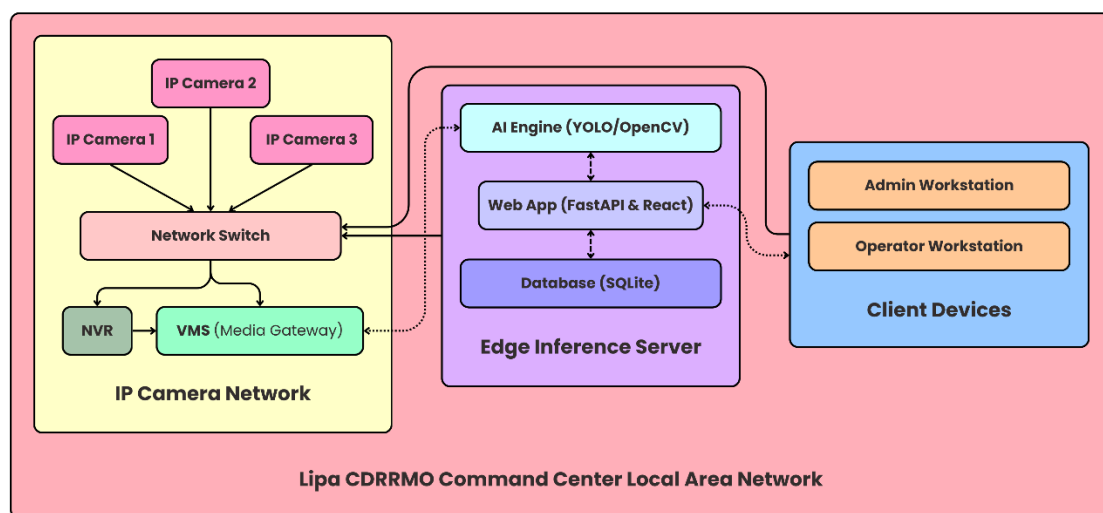


Figure 9.0. Deployment Architecture Diagram

The system utilized a localized deployment architecture based on the edge computing paradigm. Unlike traditional web applications that distribute web and database servers across remote cloud infrastructure, this system consolidated all processing onto a high-performance physical machine deployed directly within the Lipa CDRMO command center.

Client devices, which were standard computer workstations running modern web browsers, connected to the server exclusively through the internal LAN. The server simultaneously hosted the FastAPI application server, the SQLite database, the YOLO AI worker processes, and served the compiled React frontend directly to clients. This configuration eliminated network transit latency and removed the need for a separate standalone web server.

Hosting Environment

The deployment environment employed enterprise-grade hardware capable of supporting the agency's extensive video ingestion pipeline.

- **Server Specifications:** Parallel deep learning object detection and asynchronous routing for 418 cameras required an enterprise-grade edge server. The hardware specification included a Dell PowerEdge R760xa, dual Intel Xeon Platinum 8468 processors, 512GB ECC DDR5 RAM, and dual 2 TB NVMe Gen4 SSDs in RAID 1 for high-speed database input/output. The server featured a multi-GPU array of 8 NVIDIA L4 Tensor Core GPUs, collectively providing sufficient VRAM (Video Random Access Memory) to support hundreds of concurrent streams without Out-of-Memory (OOM) errors.
- **Operating System:** The edge server ran a modern 64-bit enterprise Linux distribution, such as Ubuntu Server LTS. This environment ensured kernel

stability and supported the NVIDIA CUDA Toolkit and cuDNN drivers required for sustained deep learning execution and efficient resource management.

- **Network Configuration:** The edge server connected via a Gigabit copper port (8GT) to one of the command center's Ruijie Reyee aggregation switches and was assigned a static IP address within the agency's dedicated CCTV Virtual LAN (VLAN). This setup placed the server on the same subnetwork as the VMS, specifically the Dahua DSS Pro server, enabling zero-latency intranet communication without traversing external firewalls. For client access, dispatchers used standard workstation browsers to navigate to the edge server's static IP address and designated frontend port, facilitated by cross-VLAN routing rules within the core router.

Scalability and Performance Considerations

Despite the use of enterprise-grade local hardware, processing 400+ live video feeds requires rigorous software-level optimization and resource management to maintain system stability:

- **Substream Targeting:** When requesting streams from the VMS Server, the URL included the subtype=1 parameter. This instructed the server to provide a 720p substream instead of a 2K main stream, significantly reducing inbound network traffic on the edge server and ensuring the system did not interfere with the NVRs' continuous 2K archival recording.
- **Hardware Utilization & Computational Load Balancing:** To prevent GPU thermal throttling, the AI engine limited inference to a hardware-optimized rate of 10 to 15 FPS per camera.

- **Vertical Hardware Scalability:** The architecture supported seamless vertical scaling. If additional cameras were deployed at city intersections, system capacity could be increased by installing more GPU accelerators in the edge server without modifying the underlying codebase.
- **Database Optimization:** The database was configured with WAL. This enabled the dashboard to perform intensive read operations concurrently with the AI engine's high-frequency writes. This configuration eliminated database locking errors.
- **Client-Side Caching:** To minimize redundant server queries, the React dashboard implemented TanStack Query, which temporarily cached API responses in the client browser. This approach significantly reduced backend load when multiple operators accessed analytics dashboards simultaneously.

Design Constraints and Assumptions

The deployment was subject to several operational constraints and technical assumptions:

- **Agency Infrastructure Access and Authorization:** The Lipa CDRMO IT administration authorized connection of the edge server to the core network switches and allocated a static IP address within the restricted CCTV VLAN. Furthermore, the agency provided the necessary system credentials, specifically the authorized usernames, passwords, and server IP configurations, required for the system to authenticate and request streams from the infrastructure.
- **Hardware VRAM Bottleneck:** An array of eight NVIDIA L4 GPUs supported up to 418 cameras, which constituted the maximum capacity

for simultaneous video ingestion. Any further expansion would surpass the available VRAM.

- Enterprise VMS Communication: The system interfaced directly with the Dahua DSS Pro Server. Operating as a headless client, the edge server transmitted automated RTSP requests exclusively to the VMS's internal media gateway rather than polling the physical cameras.
- Network Infrastructure: The software assumed that all target IP cameras were actively streaming standard RTSP streams and were accessible via the VMS server. Maintenance, repair, and bandwidth management of the camera infrastructure were managed independently by the agency.
- Internet Independence: If the command center's external ISP fiber connection was severed during a natural disaster, localized RTSP ingestion, AI inference, and intra-VLAN WebSocket alert deliveries remained operational.

Development Tools and Technologies

Visual Studio Code was selected as the primary Integrated Development Environment (IDE) for its lightweight architecture, integrated terminal, and extensive extension ecosystem. To maintain strict dependency locking and ensure virtual environment consistency across AI and backend workspaces, uv, a high-performance Python package manager, was employed, alongside pnpm for the frontend workspace to guarantee fast, deterministic node-module installations. For API testing and route validation during development, FastAPI's auto-generated interactive Swagger UI, alongside Postman, was used to simulate edge-server webhook payloads and RESTful operator actions.

Programming Languages

- Python (Version 3.12.13): Served as the primary language for the AI engine and backend due to its native support for both the intensive computational requirements of deep learning and the asynchronous, high-concurrency event loops required.
- TypeScript / TSX: Served as the primary language for the frontend dashboard to enforce strict static typing and interface definitions. This approach ensured that complex JSON payloads were handled predictably, significantly reducing runtime errors.
- HTML5 & CSS3: Utilized as the foundational markup and style sheet languages to define the semantic structure and visual presentation of the web interface.

Frameworks and Libraries

- Backend Layer (FastAPI): FastAPI was selected over traditional frameworks such as Django or Flask for its native asynchronous support via the ASGI specification, and is deployed using Uvicorn, a high-performance ASGI server. It offers integrated WebSocket support, facilitating efficient communication between AI workers and the user interface. Secure role-based access control (RBAC) is implemented using pyjwt for JSON Web Token (JWT) generation and passlib with bcrypt for cryptographic password hashing. Background tasks, including the automated hourly aggregation of raw hardware telemetry into historical trends, are managed by apscheduler. For hardware monitoring, psutil and gputil are employed to collect high-resolution server metrics for the system health dashboard.

- Unicorn, the ASGI server, serves as the runtime layer connecting the FastAPI application to the network. Its significance becomes evident when contrasted with its predecessor, the Web Server Gateway Interface (WSGI). Traditional WSGI servers operate on a synchronous, blocking model in which each incoming request occupies a worker thread until the response is fully delivered. Although sufficient for conventional web applications, this model creates a critical bottleneck in systems that require simultaneous, persistent connections.
- The backend is required to sustain three concurrent communication channels: persistent WebSocket connections to operator workstations, internal HTTP webhook requests from the AI engine, and standard RESTful API requests from the frontend. In a synchronous WSGI model, a long-lived WebSocket connection would monopolize a worker thread for the duration of an operator's shift, thereby limiting the server's capacity to process incoming alerts or frontend requests concurrently. ASGI addresses this limitation through Python's native asyncio event loop, which suspends idle coroutines and yields control to other pending operations instead of blocking threads. This cooperative multitasking model enables a single Unicorn worker process to maintain open WebSocket connections to all operator workstations, receive AI detection webhooks, broadcast collision alerts, and serve frontend requests within a single-process, event-driven runtime on the resource-constrained edge server.

- Frontend Layer (React.js Ecosystem & Tailwind): The web was developed with React to enable a dynamic, component-based SPA. React Router managed client-side navigation, permitting seamless transitions while maintaining persistent WebSocket connections. Zustand governed global state management for high-frequency, real-time alert payloads, enabling efficient, optimized data flows. TanStack Query was used for asynchronous REST API data fetching and caching, reducing redundant database requests at the edge server. Analytical data were visualized through the Recharts library. Tailwind CSS, shadcn/ui, and Lucide icons were employed to ensure a responsive, accessible, and cohesive web interface.
- AI Engine Layer (Ultralytics, YOLO, & OpenCV): The core collision-detection engine was developed using the Ultralytics framework, which provided an optimized Python environment for deploying the YOLO deep learning model. OpenCV was used to capture, decode, and preprocess RTSP video feeds from the IP cameras before passing the frames to the model for real-time inference.

Machine Learning Environments

- Roboflow: Utilized primarily as the central dataset management platform for bounding-box annotation and programmatic image augmentation. Its integrated environment was also leveraged for initial model training and rapid inference testing to quickly validate the dataset's efficacy before committing to full-scale compute cycles.

- Google Colab: Leveraged as the primary compute environment. Its cloud-based GPU acceleration enabled efficient training of custom YOLO weights without overloading local development hardware. It also served as a testing environment to validate the model's accuracy, precision, and recall metrics after training.
- Visual Studio Code: Used as the main development environment for integrating the finalized model weights. Real-world testing was performed here to ensure the AI engine worked smoothly with the backend and RTSP feeds.

Database Technologies

Selecting the database engine for this project required a careful evaluation of query latency, operational overhead, concurrency characteristics, and suitability for the anticipated data volumes. The primary technologies under consideration were PostgreSQL, MySQL, MongoDB, and SQLite.

PostgreSQL and MySQL are comprehensive client-server relational databases recognized for their robustness in high-concurrency enterprise environments. However, both require dedicated memory allocation and background daemons as standalone server processes, which would compete with the AI engine for the edge server's limited CPU and RAM resources. MongoDB, a document-oriented NoSQL database, was also deemed unsuitable. The project's data model is inherently relational. Enforcing these constraints in MongoDB would necessitate application-level logic to replicate relational semantics, thereby increasing code complexity without corresponding benefits.

SQLite was selected for its serverless, embedded architecture, which reads and writes directly to a single file via an in-process library, eliminating inter-process communication overhead and reducing query latency to that of a local filesystem.

operation on the edge server's storage. The projected data volumes remain well within SQLite's documented performance thresholds. The non-functional requirement to query up to 100,000 historical incident logs within three seconds was met by implementing appropriate indexing on detection status, camera identifier, and timestamp columns.

By default, SQLite's rollback journal serializes all database access during write operations, causing active writes to block readers and vice versa. This posed an operational challenge, as the system requires unavoidable read-write concurrency. Enabling WAL mode resolved this by appending changes to a separate log file, allowing readers and writers to operate concurrently. In this project, WAL ensured the AI engine could write and the frontend could read simultaneously. The WAL file is kept at a manageable size by SQLite's automatic checkpointing. This configuration was enforced via a PRAGMA statement on every database connection. Database interactions were managed using SQLModel, a modern ORM that combines SQLAlchemy and Pydantic, ensuring strict type-safety and structural validation for all archived data.

Table 9.0. Comparison of Database Engine Candidates

Feature	PostgreSQL	MySQL	MongoDB	SQLite
Architecture	Client-server	Client-server	Client-server	Serverless, embedded
Separate process required	Yes	Yes	Yes	No
Resource overhead	High	Moderate	High	Minimal
Data model	Relational	Relational	Document-oriented	Relational
Relational constraint enforcement	Native	Native	Application-level	Native
Concurrency model	MVCC	Locking	Document-level locking	WAL-enabled

Deployment complexity	High	Moderate	High	Low
Suitability for edge deployment	Low	Low	Low	High

Version Control and Collaboration Tools

Source code versioning was managed using Git to track architectural changes, prevent code conflicts, and maintain historical backups. The project was structured as a unified monorepo and hosted on GitHub, enabling collaborative development across the AI, backend, and frontend teams and providing a centralized repository for issue tracking and system documentation.

Deep Learning Implementation and Training Protocol

To achieve the 85% mAP operational threshold while maintaining sub-15-second alert propagation, the AI model underwent a strict configuration and training pipeline optimized for edge deployment. The implementation was broken down into dataset preparation, architecture selection, hyperparameter tuning, and hardware-specific compilation.

1. Dataset Preprocessing and Augmentation

The model was trained on a hybrid dataset consisting of publicly available vehicular collision imagery and custom archival footage curated from the Lipa CDRMO CCTV network. To ensure robust feature extraction and prevent model overfitting, the dataset was partitioned into a 80% Training, 10% Validation, and 10% Testing split.

- Preprocessing: All images were strictly resized to a 640x640 resolution (imgsz=640) to standardize the input tensor for the YOLO architecture.

- **Augmentation Strategy:** To address the "sim-to-real" gap and simulate Lipa City's real-world atmospheric noise, targeted augmentations were applied. This included mosaic augmentation to improve the detection of distant, small-scale vehicle collisions, as well as randomized brightness and contrast adjustments to simulate nighttime camera glare and variable weather conditions.

2. Model Architecture Selection

The YOLO26n (Nano) architecture was selected as the foundational object detection model. As established in the literature, deploying heavier, multi-stage deep learning models introduces severe latency bottlenecks. The YOLO26n variant was specifically chosen because it features an ultra-lightweight parameter count. This minimal architectural footprint easily fits within the strict VRAM limits of the localized edge server, perfectly balancing the high-speed computational requirements necessary for concurrent live RTSP stream ingestion with the 85% mAP threshold required for accurate scene classification.

3. Hyperparameter Configuration and TensorRT Optimization

The model was trained over 150 epochs to allow sufficient gradient convergence without inducing overfitting. To transition the model from a training environment to a live edge-deployment environment, the finalized PyTorch weights (.pt) were exported and compiled into a highly optimized NVIDIA TensorRT engine format (.engine).

The following execution parameters were enforced to maximize hardware utilization on the edge server's NVIDIA GPUs:

- FP16 Precision (half=True): The model weights were quantized from 32-bit floating-point down to 16-bit. This reduced the VRAM footprint by 50% and significantly accelerated inference speed without a mathematically significant drop in the target mAP.
- Dynamic Batching (dynamic=True): Enabled the inference engine to dynamically adjust the input tensor sizes at runtime, preventing memory allocation errors during variable network traffic.
- Batch Sizing (batch=8): For the prototype deployment, the batch size was explicitly set to 8. This configuration is structurally designed to scale upwards, allowing the edge server to process multiple simultaneous RTSP camera streams concurrently as the deployment expands across the city's 418-camera network.

4. Training Environment

The initial model training, dataset compilation, and weight optimization were executed within a cloud-based Jupyter Notebook environment (Google Colab). This leveraged high-performance cloud GPUs to accelerate the 150-epoch training cycle before the finalized .engine model was pulled down and deployed locally to the independent edge server for Phase 4 LAN stress testing.

Testing and Validation

Testing Strategy Overview

The testing strategy employed a phased, hybrid evaluation approach to validate both standard web application stability and computationally intensive machine learning operations. To eliminate any risk of impacting the Lipa CDRRMO's live production servers or disrupting active emergency dispatch processes, all testing was conducted exclusively within a controlled, isolated LAN staging environment. A key element of this strategy was the high-fidelity simulation of the command center's enterprise VMS architecture. Instead of evaluating the computer vision model with static, local video files, an open-source RTSP media gateway (MediaMTX) was implemented to replicate the behavior of the CDRRMO's VMS. By broadcasting benchmark traffic footage through this proxy, the AI edge server was required to ingest, decode, and process live network streams. This approach ensured that the system's network protocols, hardware limitations, and AI ingestion buffers were thoroughly validated under the precise physical constraints of the intended deployment environment, without jeopardizing the city's live operations.

Within the simulated network environment, the testing process adopted a structured, bottom-up methodology to ensure effective fault isolation. The system underwent sequential evaluation through Unit Testing, Integration Testing, AI Model Validation, Performance and Resilience Testing, System Testing, and UAT. This comprehensive approach ensured cohesive component functionality and strict compliance with all defined Functional and Non-Functional Requirements.

Types of Testing Conducted

A. Unit Testing

Unit testing formed the foundational layer of validation by isolating the smallest testable parts of the application. The objective of this phase was to ensure that individual functions, user interface elements, and database transactions executed logically before systemic integration. This phase focused on validating client-side input sanitization, backend algorithmic logic, isolated computer vision utility functions, and local database concurrency safeguards.

B. Integration Testing

After unit validation, integration testing assessed the communication pathways and data exchange throughout the system architecture. The primary objective was to verify the integrity of the data pipeline. This phase examined the network handshake for video ingestion, telemetry transfer from the AI worker to the backend application, and bi-directional communication between the REST/WebSocket APIs and the React client, ensuring no data loss across module interfaces.

C. AI Model Validation

Given the system's reliance on an autonomous computer vision engine, specialized validation was performed to evaluate the machine learning model based solely on its algorithmic performance. The objective was to establish quantitative baseline metrics for the computer vision pipeline. This phase assessed the model's spatial accuracy, robustness to environmental visual occlusion (e.g., weather or low light), and the optimization of confidence thresholds to balance true positive detection with the risk of alert fatigue.

D. Performance and Resilience Testing

This phase subjected the system to controlled operational stress and network instability to validate the application's physical architecture. The objective was to ensure strict adherence to defined NFRs. Testing addressed

network ingestion limits, disaster recovery protocols during simulated hardware failures, asynchronous state synchronization, and long-term endurance to confirm the system's viability for continuous 24/7 command center deployment.

E. System Testing

System testing assessed the fully integrated application as a single cohesive entity. The objective was to validate the complete operational lifecycle and technical workflows from end to end prior to user deployment. This phase emphasized the execution of the uninterrupted HITL pipeline (from network video ingestion to UI alert resolution), the enforcement of RBAC routing protocols, and the validation of system-wide utility functions (e.g., historical data filtering and formatted report generation).

Test Cases

A. Unit Testing

Table 9.1. Frontend UI Components

Test ID	Module	Scenario	Pre-Conditions	Expected Result
TC-U-101	Authenti- cation	Login Form Input Sanitization	User is on the Login page.	The login form strictly rejects SQL injection strings (e.g., ' OR 1=1;--) and disables the submit button if fields are empty.
TC-U-102	User Manage- ment	Password Complexity Validation	Admin is creating a new user account.	The UI immediately flags passwords failing complexity rules (e.g., < 8 chars, missing numbers) without needing a server response.
TC-U-103	RBAC UI	Role-Base d UI Rendering (Operator)	State is initialized with a standard Operator JWT token.	The "User Management" tab is entirely removed from the React DOM and does not render in the sidebar navigation.

TC-U-104	Analytics UI	Historical Log Date Filter Logic	User selects a Start Date and End Date in the Analytics tab.	The UI local state prevents selecting an End Date that chronologically occurs before the Start Date.
TC-U-105	Dashboard Alerts	HITL Workflow Click Constraint	An active alert modal is populated with mock snapshot data.	The UI strictly requires no more than 2 clicks to hit 'Confirm' or 'Dismiss', and successfully renders the 'Mute/Snooze' toggle on the active alert modal
TC-U-106	Help Center	Markdown Rendering	User navigates to the Help Center.	The React markdown parser correctly renders headings, lists, and bold text for the local SOP documentation.
TC-U-107	User Management	Password Confirmation Match	User inputs values into "New Password" and "Confirm Password" during account creation or editing.	The UI disables the submit button and displays a validation error if the two password fields do not perfectly match.
TC-U-108	Alarm Settings	Snooze Duration Input Validation	User attempts to input 10 (below minimum) and 200 (above maximum) as the snooze duration.	The UI rejects both values, clamps or flags them, and enforces the 15–60 second bounds without needing a server response.

Table 9.2. Backend API and Auth Middleware

Test ID	Module	Scenario	Pre-Conditions	Expected Result
TC-U-201	Security	Password Hashing Algorithm	A mock plaintext password is submitted to the utility function.	The hashing function outputs a salted bcrypt/argon2 string, and the verifier function successfully matches the plain text to the hash.

TC-U-202	Session Auth	JWT Generation	User logs in successfully (simulated).	The backend generates a JWT containing the correct User ID and Role payload, with a strict expiration timestamp.
TC-U-203	Session Auth	JWT Expiration Handling	An expired JWT token is passed to the Auth middleware.	The middleware strictly rejects the token, throws a 401 Unauthorized exception, and does not execute the protected function.
TC-U-204	Route Guard	RBAC Route Protection Logic	A valid token with the role "Operator" is passed to a User Management endpoint middleware.	The middleware intercepts the request and instantly throws a 403 Forbidden exception without processing the payload.
TC-U-205	WebSoc ket API	WebSocket Payload Formatting	A simulated collision detection dictionary is passed to the formatter function.	The function successfully outputs a strict JSON schema that maps directly to the initial Detection Log record, containing exactly: log_id, camera_id, detected_at, confidence_score, snapshot_path, and an initial detection_status of "Unverified".

Table 9.3. AI Utility and Data Processing

Test ID	Module	Scenario	Pre-Conditions	Expected Result
TC-U-301	Ingestion Buffer	FPS Downsampling Logic	A simulated array of 30 sequential video frames is passed to the buffer utility.	The utility array slicing correctly drops intermediate frames, outputting exactly 10 to 15 frames to match the hardware optimization rule.
TC-U-302	Inferenc e Logic	Confidence Score Thresholdin g	YOLO mock outputs [0.45, 0.88, 0.60] against a set threshold of 0.75.	The filtering function strips out the 0.45 and 0.60 values, returning only the 0.88 detection object.

TC-U-303	Data Parser	Bounding Box Coordinate Extraction	A mock YOLO tensor object is passed to the parser.	The function accurately extracts the [x, y, width, height] pixel coordinates and translates them into a standardized Python dictionary.
----------	-------------	------------------------------------	--	---

Table 9.4. Database Concurrency and Query Logic

Test ID	Module	Scenario	Pre-Conditions	Expected Result
TC-U-401	Incident CRUD	Default Incident Status Insertion	The CRUD function inserts a new collision record without a specified status.	The database schema successfully defaults the status column to strictly read "Unverified".
TC-U-402	Concurrency	WAL Concurrency	A Python script attempts to write an incident log at the exact millisecond a separate script reads the analytics table.	The WAL mechanism successfully queues the transaction; neither script throws a "Database is locked" fatal error.
TC-U-403	Audit Logging	Audit Trail Timestamp Generation	The CRUD function updates an incident from "Unverified" to "Ongoing".	The database correctly appends the verified_at and verified_by fields without overwriting the original detected_at timestamp.
TC-U-404	DB Maintenance	Telemetry Pruning Utility	The cleanup utility is executed on a mock database containing telemetry logs from 72 hours ago.	The script correctly executes a DELETE SQL statement targeting only logs older than 48 hours, leaving recent logs untouched.
TC-U-405	Data Aggregation	Telemetry Aggregation Math	The hourly cron job function reads 12	The utility accurately calculates the mean average of those 12 points and formats it for the

			high-resolution 5-minute telemetry data points.	low-resolution 30-day retention table.
--	--	--	---	--

B. Integration Testing

Table 10.1. Video Stream Ingestion

Test ID	Module	Scenario	Pre-Conditions	Expected Result
TC-I-101	RTSP Client	Media Gateway Handshake Validation	MediaMTX is broadcasting a 720p substream. AI engine initiates a connection.	The AI engine successfully establishes a TCP connection, parses the SDP (Session Description Protocol), and begins receiving video packets without a timeout.
TC-I-102	RTSP Client	Substream Parameter Enforcement	The AI engine requests a stream using subtype=1 appended to the URL.	The ingestion engine confirms the incoming stream resolution is compressed 720p rather than the 4K main stream, preventing bandwidth flooding.
TC-I-103	Ingestion Buffer	Live Frame Decoding to OpenCV	RTSP packets are being actively received by the AI engine.	The cv2.VideoCapture module successfully decodes the raw network packets into usable NumPy arrays (frames) without corruption or "gray frame" tearing.
TC-I-104	Stream Controller	Multi-Stream Initialization	The database contains 3 active camera URLs. The master node boots up.	The AI engine successfully spawns 3 asynchronous ingestion threads, establishing independent RTSP handshakes for each camera simultaneously.

Table 10.2. AI Telemetry Handover

Test ID	Module	Scenario	Pre-Conditions	Expected Result
---------	--------	----------	----------------	-----------------

TC-I-201	Telemetry Pipeline	YOLO Bounding Box Serialization	The YOLO thread detects a collision and outputs raw Python tensor data.	The pipeline successfully serializes the multidimensional array to the standard JSON format and passes it to the AI engine main thread without data type mismatches.
TC-I-202	State Controller	AI Pause Signal Handover	An Operator confirms an alert via the UI. FastAPI flags the camera state as "Ongoing".	The backend successfully passes the "Pause" boolean flag to the isolated AI worker thread, which instantly ceases frame ingestion until resolved.
TC-I-203	State Controller	Cooldown Signal Enforcement	An Operator dismisses an alert. FastAPI triggers the 1-minute cooldown.	The backend passes the timestamp to the AI worker; the worker correctly rejects all subsequent detections on that specific stream until the 60 seconds expire.
TC-I-204	Hardware Telemetry	System Health Handover	The master node is running normally.	The OS utilities successfully pull CPU/GPU metrics and pass them to the FastAPI router, which formats them into the system health payload.

Table 10.3. API & Data Persistence

Test ID	Module	Scenario	Pre-Conditions	Expected Result
TC-I-301	WebSocket API	Real-Time Alert Dispatch	A collision is logged to the DB. A client is logged into the React dashboard.	FastAPI instantly pushes the collision JSON payload over the WebSocket connection; the React UI receives it, synchronously triggers the visual alert modal, and initiates the configured auditory alarm.
TC-I-302	Incident CRUD	Historical Data Query and Hydration	User navigates to Analytics and requests logs	React sends the GET request; FastAPI queries SQLite, retrieves the data array, and React

			from the past 7 days.	successfully parses the response into the table UI.
TC-I-303	Camera CRUD	Adding a New RTSP Node	User submits a new Camera configuration via the React modal.	API receives the POST request, validates the payload, commits the new row to SQLite, dynamically spawns the background RTSP ingestion and YOLO inference threads for the new feed, and React updates the Camera List state.
TC-I-304	Camera CRUD	Disabling an Active Camera	User toggles a camera status to "Disconnected" in the UI.	API updates the DB status, sends a kill signal to the specific camera's ingestion thread, and the UI immediately reflects the "Disconnected" status.
TC-I-305	Analytics API	Dashboard KPI Aggregation	The dashboard loads and requests top-level KPIs (e.g., Total Accidents, False Positives).	FastAPI successfully executes COUNT and GROUP BY SQL queries, calculates the totals, and transmits the aggregated numbers to the React frontend.
TC-I-306	Alarm Settings CRUD	Alarm Config Save and Retrieve	Operator saves a new sound selection, volume, and snooze duration.	API commits the config to the database and a subsequent GET request returns the exact same values, confirming persistence.
TC-I-307	Snooze Signal Handover	Per-Camera Snooze Activation	Operator clicks snooze on Camera A's alert.	The backend receives the snooze signal, mutes the alarm for Camera A's alert specifically, and starts the countdown timer. Camera B fires an alert simultaneously and its alarm plays unaffected.
TC-I-308	Snooze Signal Handover	Snooze Re-Alarm Trigger	Snooze is active on an unresolved Camera A alert. The configured	The backend timer fires, the WebSocket pushes a re-alarm signal to the client, and the alarm sounds again for that specific alert.

			snooze duration expires.	
--	--	--	--------------------------	--

Table 10.4. Audit Trail & Security Synchronization

Test ID	Module	Scenario	Pre-Conditions	Expected Result
TC-I-401	Audit Logging	Operator Incident Resolution Verification	Operator clicks "Confirm" on an active alert using a valid JWT token.	The API intercepts the JWT, extracts the user ID, updates the SQLite incident status to "Ongoing", and permanently logs the user ID in the verified_by column.
TC-I-402	User CRUD	Admin Account Creation Sync	Admin creates a new Operator profile via the UI.	API receives the request, securely hashes the password, writes the new user row to SQLite, and returns a 201 Created response to the UI.
TC-I-403	System Export	Historical Data CSV Generation	The user applies date/camera filters on the dashboard and initiates the "Export Reports" function, selecting their desired output format	The FastAPI backend successfully queries the SQLite database using the active filters, dynamically transforms the dataset into the requested file format (e.g., CSV, PDF), and securely streams the generated file payload back to the client for download.
TC-I-404	Auth Middleware	Forced Session Logout (Token Revocation)	An Operator clicks "Log Out" on the React dashboard.	The UI destroys the local token, notifies the backend to close the user's active WebSocket connections, and then successfully redirects to the login screen.

C. AI Model Validation

Table 11.1. Algorithmic Accuracy & Spatial Metrics

Test ID	Module	Scenario	Pre-Conditions	Expected Result
TC-AI-101	Object Detection	Baseline True Positive Detection	AI processes a benchmark video of a clear, daylight vehicle collision.	The YOLO model successfully identifies the collision, achieving an operational accuracy that meets or exceeds the strictly defined 85% mAP and 0.50 IoU baseline thresholds.
TC-AI-102	Spatial Logic	Bounding Box Localization (IoU)	AI processes a collision frame with a known ground-truth coordinate box.	The generated bounding box achieves an Intersection over Union (IoU) score of > 0.50, proving it accurately surrounds the actual vehicles.
TC-AI-103	Temporal Logic	Sustained Incident Tracking	AI processes a 10-second clip of two crashed cars remaining stationary.	The model consistently outputs bounding box coordinates across the entire 10-second sequence without the detection "flickering" or dropping.

Table 11.2. Environmental Robustness

Test ID	Module	Scenario	Pre-Conditions	Expected Result
TC-AI-201	Environmental	Low-Light / Nighttime Inference	AI processes benchmark footage of a nighttime collision under standard streetlights.	The model successfully detects the collision without requiring infrared/thermal imaging inputs.
TC-AI-202	Environmental	Heavy Rain / Noise Occlusion	AI processes footage of an intersection during heavy rainfall.	The model filters out the visual noise (raindrops/puddles) and successfully triggers a true positive detection.

TC-AI-203	Environmental	Lens Flare / Headlight Glare	Nighttime footage featuring high-beam headlights pointed directly at the camera lens.	The model correctly ignores the optical anomaly and does not hallucinate a false positive bounding box.
TC-AI-204	Spatial Occlusion	Partial Line-of-Sight Blockage	A collision occurs, but 30% of the incident is physically blocked by a passing truck or tree branch.	The model interpolates the visible vehicle structures and maintains a successful detection despite the occlusion.

Table 11.3. Precision Calibration

Test ID	Module	Scenario	Pre-Conditions	Expected Result
TC-AI-301	Threshold Logic	Sub-Threshold False Positive Suppression	System threshold is 75%. Video shows a "near-miss" or heavy braking, yielding a 60% AI confidence score.	The threshold filter successfully intercepts the detection and drops it, preventing the system from alerting the dashboard.
TC-AI-302	Algorithmic Bias	Pedestrian/Motorcycle Exclusion	Video features high-density pedestrian crossing and motorcycle traffic with no collision.	The YOLO model strictly isolates vehicular data and generates 0 false positive alerts regarding standard human/motorcycle movement.

Table 11.4. Hardware Profiling & Compute Optimization

Test ID	Module	Scenario	Pre-Conditions	Expected Result
---------	--------	----------	----------------	-----------------

TC-AI-401	Inference Engine	Strict Frame Latency Limit	The YOLO model is actively processing standard 720p substream frames.	The GPU completes the mathematical forward pass and outputs bounding box coordinates in strictly under 100 milliseconds per frame.
TC-AI-402	Ingestion Pipeline	Target FPS Maintenance	The AI worker processes a continuous 5-minute RTSP feed.	System telemetry confirms the AI engine is actively evaluating a steady 10 to 15 frames per second, maintaining real-time parity without bottlenecking.
TC-AI-403	Hardware Limits	VRAM Allocation Stability	The edge server is initialized with 3 concurrent camera streams routed to the GPU.	The YOLO tensor allocation remains stable; the GPU VRAM does not exceed physical hardware limits or trigger an OOM crash.

D. Performance and Resilience Testing

Table 12.1. VMS Proxy Simulation

Test ID	Module	Scenario	Pre-Conditions	Expected Result
TC-R-101	Network Ingestion	Proxy Handshake Integrity	MediaMTX proxy is running. Master node initiates a connection request.	The AI engine successfully bypasses direct camera IPs, resolving the RTSP stream strictly through the MediaMTX proxy server port.
TC-R-102	Bandwidth Throttling	Multi-Stream Bandwidth Constraint	System initializes 5 concurrent simulated video streams via the proxy.	The edge server's Network Interface Card (NIC) handles the continuous packet influx without dropping frames or triggering TCP timeout errors.

Table 12.2. Concurrency & Load Testing

Test ID	Module	Scenario	Pre-Conditions	Expected Result
---------	--------	----------	----------------	-----------------

TC-R-201	Async Processing	Strict UI Alert Latency (< 2s)	AI detects a collision. A stopwatch is initiated at the moment of detection.	The JSON payload traverses the database and WebSocket, triggering the React UI alert modal in strictly under 2.0 seconds.
TC-R-202	Database Indexing	High-Volume Query Latency (< 3s)	SQLite database is populated with 100,000 mock incident rows. User executes a date-filtered query.	The system successfully filters the dataset and renders the analytical table in the React UI in strictly under 3.0 seconds.
TC-R-203	Data Export	Report Generation Latency (< 5s)	User requests a 30-day historical data export comprising approximately 10,000 records, selecting their desired output format (e.g., CSV, PDF).	The FastAPI backend successfully compiles the requested file payload and initiates the browser download prompt in strictly under 5.0 seconds.

Table 12.3. Disaster Recovery, Isolation & Fault Tolerance

Test ID	Module	Scenario	Pre-Conditions	Expected Result
TC-R-301	Fault Tolerance	RTSP Network Disconnect & Recovery	System is actively monitoring a stream. The network connection to MediaMTX is digitally severed.	The UI flags the camera as "Disconnected" The AI engine initiates an automated reconnection loop every 10 seconds until the stream is restored.
TC-R-302	Thread Management	Process Isolation (Fatal AI Error)	A deliberate Python exception is injected into Camera 1's	Camera 1's thread crashes safely. The FastAPI application and the parallel AI threads for Cameras 2 and 3 remain entirely

			YOLO worker thread.	unaffected and continue processing.
TC-R-303	System Maintenance	24-Hour Automated Restart Protocol	Server clock strikes the designated low-traffic window (e.g., 3:00 AM).	The system safely flushes system memory, automatically restarts the master application services, and resumes active network ingestion in under 10 seconds.
TC-R-304	State Recovery	Asynchronous Browser Crash	An alert is active on the screen. The operator accidentally force-closes the browser tab.	Upon reopening the browser and restoring the secure session, the UI instantly queries the backend and repopulates the "Unverified" alert, ensuring zero data loss.

Table 12.4. Endurance Testing

Test ID	Module	Scenario	Pre-Conditions	Expected Result
TC-R-401	Memory Management	24-Hour Continuous Execution (RAM)	Master node runs 3 continuous simulated streams for exactly 24 hours.	AI engine garbage collection actively clears old video buffers; standard system RAM utilization remains flat and does not exhibit a continuous upward memory leak.
TC-R-402	Hardware Thermal s	24-Hour GPU Thermal Stability	The YOLO worker continuously processes 15 FPS across multiple streams for 24 hours.	The GPU thermal telemetry remains within safe operational limits (avoiding thermal throttling), and VRAM allocation remains rigidly locked without bloat.

E. System Testing

Table 13.1. End-to-End HITL Workflow

Test ID	Module	Scenario	Pre-Conditions	Expected Result
---------	--------	----------	----------------	-----------------

TC-S-101	E2E Pipeline	Full True Positive Resolution	MediaMTX streams a collision. Operator is logged into the dashboard.	AI engine detects the event → FastAPI logs to SQLite → WebSocket triggers UI alert with audible alarm → Operator clicks "Snooze"; the alarm for that specific camera mutes and the configured countdown timer begins → Operator reviews the snapshot and clicks "Confirm" before the timer expires → The system correctly flags the event as "Ongoing," and pauses AI ingestion for that camera.
TC-S-102	E2E Pipeline	Full False Positive Dismissal	AI incorrectly flags an event. Operator receives the active alert.	Audible alarm sounds for the flagged camera → Operator clicks "Snooze"; the alarm for that specific camera mutes and the countdown timer begins → Operator reviews the snapshot, determines it is a false positive, and clicks "Dismiss" before the timer expires → The UI clears the alert from the queue, the DB logs the event as "Dismissed," and the backend enforces the 1-minute AI cooldown on that specific camera.
TC-S-103	Workflow Limits	Operational Efficiency Timing (<15s)	A collision occurs on the simulated camera feed.	The Operator is able to visually identify the UI alert, verify the snapshot, and click "Confirm" in strictly under 15 seconds from the moment of actual detection.
TC-S-104	UI Audio/Visual	Command Center Alert Distinctiveness	Dashboard is open in the browser.	An incoming alert successfully triggers a distinct, sustained audible alarm and forces a visual state change (e.g., modal popup) that captures immediate user attention.

TC-S-105	Workflow Limits	Click Constraint Verification (<=3 Clicks)	Multiple alerts populate the dashboard queue.	The Operator can successfully review a snapshot and execute a resolution ("Confirm" or "Dismiss") using a maximum of 3 mouse clicks per incident.
----------	-----------------	--	---	---

Table 13.2. System Security & Route Protection

Test ID	Module	Scenario	Pre-Conditions	Expected Result
TC-S-201	Security Routing	Operator Route Blocking	An Operator manually types /admin/users into the browser URL bar.	The React router intercepts the navigation, blocks access to the User Management DOM, and redirects the Operator back to the primary dashboard.
TC-S-202	Security Routing	Admin Route Authorization	An Administrator logs into the dashboard and clicks the "User Management" tab.	The React router permits the navigation, the FastAPI backend validates the Admin JWT, and the system successfully renders the restricted user directory.
TC-S-203	Session Mgmt	Persistent Standby State	Operator logs in and leaves the dashboard idle for 4 hours.	The JWT token successfully maintains the secure session. When a simulated alert is fired at hour 4, the WebSocket pushes the payload without requiring the user to re-authenticate.

Table 13.3. System Utility, Analytics, & Export

Test ID	Module	Scenario	Pre-Conditions	Expected Result
TC-S-301	Data Querying	Complex Multi-Parameter Filtering	User navigates to the Detection Logs table.	The user successfully applies overlapping filters (e.g., Date = "Yesterday", Status = "Confirmed", Camera = "SM Lipa"), and the UI correctly renders the precise intersection of that data.

TC-S-302	Report Export	Formatted PDF/CSV Generation	User applies a custom date filter and clicks "Export".	The system generates a downloadable file that strictly matches the filtered dataset, cleanly formatted with appropriate headers for official inter-agency record-keeping.
TC-S-303	Help Center	RBAC Documentation Rendering	Both an Admin and an Operator log in on separate machines and open the Help Center.	The Operator strictly sees operational SOPs (e.g., "How to Confirm Alerts"), while the Admin interface dynamically reveals additional IT documentation (e.g., "How to Reset User Passwords").

Table 13.4. UI Stability

Test ID	Module	Scenario	Pre-Conditions	Expected Result
TC-S-401	UI Queuing	Concurrent Alert Stacking	3 separate simulated cameras trigger collision detections at the exact same millisecond.	The React frontend successfully queues incoming WebSocket payloads, stacking the visual alerts logically in the UI without crashing the DOM or dropping any alerts.
TC-S-402	UI Queuing	Concurrent Alerts in Mixed Snooze States	Camera A is snoozed, Camera B fires a new alert simultaneously.	React frontend correctly reflects Camera A as snoozed (alarm muted) and Camera B as active (alarm sounding) without state bleeding between the two alert instances.

Table 14.1. Activity Audit Trail Testing

Test ID	Module	Scenario	Pre-Conditions	Expected Result
---------	--------	----------	----------------	-----------------

TC-S-501	Audit Logging	Incident Action Audit	An Operator is logged in and receives an active accident alert.	The system records the Operator's confirm, dismiss, resolve, or correction action with the user ID, affected incident, timestamp, and result.
TC-S-502	Audit Logging	Camera Configuration Audit	An Operator adds, edits, disables, enables, or removes a camera record.	The system records the camera action with the user ID, affected camera, timestamp, and result.
TC-S-503	Audit Logging	Report Generation Audit	An Operator exports incident records or generates a report.	The system records the export action with the user ID, selected filters or date range, timestamp, and result.
TC-S-504	Security Audit	Login Attempt Audit	A user attempts to log in using valid or invalid credentials.	The system records successful and failed login attempts with the username or user ID, timestamp, and result.
TC-S-505	Security Audit	Administrator Account Audit	An Administrator creates, edits, disables, changes a role, or updates a user password.	The system records the account-management action with the Administrator ID, affected user account, timestamp, and result.

User Acceptance Testing

UAT was conducted to validate the system's operational lifecycle, interface intuitiveness, and alignment with established emergency response workflows. The UAT scope included the entire system architecture, ensuring that real-time dispatch monitoring, system configuration, and analytics met the practical requirements of end users.

To ensure high-fidelity feedback across all system modules, UAT participants were selected directly from the Lipa CDRRMO Operations and Warning Division. The evaluation group was divided into two distinct roles to test the system's RBAC and complete feature set:

- Operators: Personnel responsible for monitoring the command center's video wall. This group tested real-time HITL alert resolution, camera stream configuration, and historical data export functions, as operators are the primary users of these features.
- Administrators: Senior division members responsible for system security. This group exclusively evaluated restricted administrative modules, including user account management and permission oversight.

To maintain safety protocols, UAT was not conducted on the CDRRMO's live production servers. Testing was performed within an isolated LAN staging environment, simulating the command center's Dahua DSS Pro server's media gateway.

The UAT procedure followed a structured, role-based methodology to evaluate the entire system:

1. Operator Scenarios (Monitoring & Configuration): Dispatchers performed standard monitoring duties using the system dashboard. Evaluation criteria included the ability to recognize alerts, review generated snapshots, and confirm or dismiss simulated traffic collisions. Additionally, dispatchers configured the system by adding a simulated intersection using an RTSP stream URL.
2. Operator Scenarios (Analytics & Reporting): Dispatchers utilized the system's historical data features. Evaluators navigated logs, applied date and location

filters, and generated CSV/PDF incident reports to validate the system's effectiveness for post-incident record-keeping.

3. **Operator Scenarios (System Health & AI Diagnostics):** Dispatchers evaluated infrastructure dashboards. For system health, evaluators reviewed real-time telemetry, including simulated GPU and CPU thermal metrics, RAM utilization, and RTSP network latency, to validate hardware monitoring capabilities. For AI performance, evaluators interacted with the AI performance module, reviewing YOLO inference speeds (FPS), operator correction rates (false positive tracking), and confidence score trends across specific camera nodes.
4. **Administrative Scenarios (Security Management):** Supervisors executed restricted system configurations, including creating new operator accounts, modifying existing credentials, and verifying that standard operator accounts were denied access to user management functions.
5. **Post-Evaluation Assessment:** After completing the scenarios, all participants assessed the system using a standardized tool to quantify usability across all modules.

The comprehensive UAT assessment gathered qualitative and quantitative feedback across four critical domains representing the entire system:

- **Workflow Integration & Interface Intuitiveness:** The assessment evaluated the dashboard's learning curve, determining whether both the alert interface and configuration settings enabled operators to navigate effectively without extensive technical training or disruption to existing workflows.
- **Alert Perception and Clarity:** The assessment measured the effectiveness of the UI's alert mechanisms, ensuring that auditory and visual prompts captured dispatcher attention without causing alert fatigue. It also verified that the

bounding box and timestamp data provided sufficient context for emergency deployment.

- **Configuration Efficiency:** The assessment validated the ease of system maintenance. Operators evaluated the simplicity of adding or modifying RTSP camera stream connections, while administrators assessed the user management interface for creating and securing employee credentials.
- **Data Utility and Reporting:** The assessment evaluated the accuracy and formatting of the system's exportable reports (CSV/PDF) to determine whether the exported historical logs met the administrative standards required by the CDRRMO for inter-agency coordination.
- **Diagnostic Clarity & Infrastructure Visibility:** The assessment validated the effectiveness of the system health and performance dashboards. Operators evaluated whether data visualizations (e.g., hardware temperature charts and AI accuracy graphs) accurately translated raw backend telemetry into actionable and comprehensible metrics.

Deployment and Implementation

The deployment phase transitions the system from a controlled development environment into a fully operational edge-computing solution within the Lipa CDRRMO. As established in the system architecture, the deployment is strictly localized; the enterprise-grade inference server (equipped with NVIDIA L4 GPUs) is physically installed within the command center and integrated directly into the agency's existing CCTV VLAN to ensure zero-latency communication with the Dahua VMS.

To ensure a seamless integration that does not disrupt ongoing emergency monitoring operations, the implementation follows a structured, multi-step strategy.

Installation and Configuration Procedures

The physical and software installation is executed in a strict sequence to guarantee system stability:

- **Hardware & Network Initialization:** The edge server is rack-mounted and connected to the command center's core aggregation switches. IT administrators assign a static IP address within the restricted CCTV VLAN, establishing the necessary cross-VLAN routing rules to allow standard Operator workstations to access the web dashboard.
- **Environment Setup:** The server is configured with an enterprise Linux distribution (Ubuntu Server LTS), followed by the installation of the NVIDIA CUDA Toolkit and cuDNN drivers necessary for hardware-accelerated deep learning.
- **Application Deployment:** The system's software stack (FastAPI backend, React frontend, and AI engine) is deployed. The local SQLite database is initialized with WAL enabled to handle high-frequency concurrent writes.
- **VMS Integration:** The system is securely authenticated as a headless client to the Dahua DSS Pro server, configuring it to passively request 720p substreams rather than pulling massive 2K main streams.

Phased Implementation Strategy

To mitigate risk and prevent overwhelming both the hardware network and the human dispatchers, the system utilizes a Phased Rollout Strategy:

- **Phase 1 (Pilot Deployment):** The system is initially activated to monitor only 10 to 15 known high-risk intersections in Lipa City. This allows developers to monitor real-world GPU thermal loads, VRAM stability, and inference latency under live conditions.

- Phase 2 (Gradual Scaling): Once the pilot phase demonstrates sustained stability and the dispatchers become comfortable with the alert frequency, the system incrementally ingests additional camera streams until the entirety of the Lipa CDRRMO's 418-camera network is fully integrated and actively monitored by the edge server.

Data Migration and Initialization

Because this is a novel system, there is no legacy AI detection data to migrate. However, an initial data seeding process is required. During deployment, the geographical and technical metadata of the city's active cameras (i.e., camera names, channel IDs) are exported from the Dahua VMS and systematically migrated into the system database. This ensures the dashboard's camera management list accurately reflects the city's physical infrastructure from day one.

User Training and Handover

Technical deployment must be paired with operational readiness. Formal training sessions are conducted to transition the system to the CDRRMO staff:

- Operator Training: Dispatchers are trained on the HITL workflow, specifically how to rapidly evaluate, confirm, or dismiss alerts to maintain a sub-15-second response time. They are also instructed on generating historical reports and reading system health telemetry.
- Administrator Training: Command center supervisors are trained exclusively on the secure user management directory, learning how to provision new dispatcher accounts, enforce access control, and manage passwords.

Maintenance and Support Plan

To guarantee the long-term viability of the command center solution, a structured maintenance protocol is established:

- **Infrastructure Monitoring:** Continuous tracking of system-wide hardware telemetry to proactively identify and mitigate performance bottlenecks, resource exhaustion, and critical system errors.
- **AI Model Retraining (Mitigating Data Drift):** As environmental conditions change or new vehicles enter the local roadways, the YOLO model may experience "data drift." A scheduled maintenance plan includes periodically exporting the locally validated incident snapshots (True Positives and False Positives categorized by the Operators) to retrain and refine the model weights, continuously improving the AI's accuracy specifically for Lipa City's traffic conditions.
- **Disaster Recovery Protocols:** To guarantee operational resilience against catastrophic hardware failures or severe logical corruption, the Administrators are required to execute a manual "Bi-Annual Restore Drill." This proactive maintenance protocol involves manually download the scheduled database backup and its associated visual snapshots from the air-gapped Network Attached Storage (NAS). Supervisors must then deploy this backup into a secure, localized staging environment entirely isolated from the production network. This drill strictly validates the integrity of the archived data, verifies the efficacy of the "Flag and Restart" restoration architecture, and ensures that agency personnel maintain the technical readiness required to execute a rapid system recovery during a critical emergency.